



河南大學  
HENAN UNIVERSITY

# 第5章 中央处理器

---

|                           |      |
|---------------------------|------|
| <u>5.1 CPU的功能和组成</u>      | ——掌握 |
| <u>5.2 指令周期</u>           | ——掌握 |
| <u>5.3 时序发生器和控制方式</u>     | ——了解 |
| <u>5.4 微程序控制器</u>         | ——掌握 |
| <u>5.5 硬连线控制器</u>         | ——了解 |
| <u>5.6 流水线技术与流水处理器CPU</u> | ——理解 |

## >>> 5.1.1 CPU的功能

### ➤ CPU（中央处理器）

- 控制程序按设定方式执行；

### ➤ CPU的主要功能：

- 指令控制 **顺序寻址、跳跃寻址**
  - ◆ 控制程序的执行顺序；
- 操作控制 **对指令操作码译码后产生控制信号**
  - ◆ 产生和发送各部件的操作控制信号；
- 时间控制 **指令各个操作实施时间的定时**
  - ◆ 控制指令、或操作的实施时间和先后顺序；
- 数据加工 **由ALU完成具体的运算**
  - ◆ 对数据进行算术逻辑运算；

## >>> 5.1.2 CPU的基本组成

### ➤现代的CPU的组成

- 运算器、控制器、片内Cache;

冯·诺依曼机的定义

### ➤控制器

- 性质：控制部件
- 组成：程序计数器、指令寄存器IR、指令译码器ID、时序发生器和操作控制器
- 功能
  - ◆取指：从内存中取出一条指令，并指出下条指令的存放位置；
  - ◆分析：对指令进行译码，产生相应的操作控制信号；
  - ◆执行：控制CPU、内存和输入/输出设备之间数据流动；

## >>> 5.1.2 CPU的基本组成

---

### ➤ 运算器

- **性质：** 执行部件
- **组成：** 算术逻辑运算单元(ALU)、寄存器组、状态条件寄存器
- **功能：** 数据加工
  - ◆ 执行所有的算术运算；
  - ◆ 执行所有的逻辑运算，并进行逻辑测试。

## >>> 5.1.3 CPU中的主要寄存器 (1/3)

➤ 中央处理器**CPU** = 运算器 + **cache**  
+ 控制器

➤ 运算器

- **ALU**

- **通用寄存器**: R0 ~ R3 (所用模型中)

- **暂存器**: DR

- **程序状态字寄存器**: PSWR

➤ **cache**

- **指令cache**: PC, IBUS

- **数据cache**: AR, DBUS

➤ **数据缓冲寄存器 (DR)**

- 暂时存放运算器与外界传送的数据。

- **作用**:

- ① 作为CPU和内存、外部设备之间信息传送的中转站;

- ② 补偿CPU和内存、外围设备之间的速度差异;

➤ **指令寄存器 (IR)**

- 保存当前正在执行的一条指令。

- 其对应的操作码字段直接作为**指令译码器**的输入。CPU中的通用寄存器可多达16个, 32个, 甚至更多。

## >>> 5.1.3 CPU中的主要寄存器 (2/3)

### ➤ 程序计数器 (PC)

- 始终存放下一条指令的地址，对应于指令Cache的访问；
- 其内容变化分两种情况
  - ◆ 顺序执行：  $PC+1 \rightarrow PC$
  - ◆ 转移执行： (指令OPR)  $\rightarrow PC$

计数自增功能

### ➤ 地址寄存器 (AR)

- 保存当前CPU所访问数据的内存单元地址；
- 上图中显示AR中的地址用于寻址数据Cache中的单元；
  - ◆ 对于数据Cache未命中的情况下，AR中的内容将用于访问CPU之外的存储器。

### ➤ 通用寄存器

- 功能：暂时存放ALU运算的数据或结果。

### ➤ 程序状态字寄存器/状态条件寄存器 (PSWR)

- 保留算术逻辑运算指令或测试指令的结果而建立的各种状态信息
- 保存各种状态和条件控制信号；
  - ◆ 进位标志(C)，溢出标志(V)，零标志(Z)，符号标志(N)
- 每个信号由一个触发器保存，从而拼成一个寄存器。

# >>> CPU中的主要寄存器

## 1. 数据缓冲寄存器（DR）

- 暂时存放运算器与外界传送的数据。
- 补偿CPU和主存、外围设备的速度差；

## 2. 通用寄存器

- 暂时存放ALU运算的数据或结果。
- 一般包含16/32个，甚至更多寄存器；

## 3. 状态条件寄存器（PSW）

- 保存各种状态和条件控制信号；
  - ◆进位标志，溢出标志，零标志，符号标志；
  - ◆保存中断和系统工作状态等信息

## 4. 地址寄存器（AR）

- 保存当前CPU要访问的主存单元地址；

## 5. 程序计数器（PC）

- 始终存放下一条指令的地址，对应于指令Cache的访问；

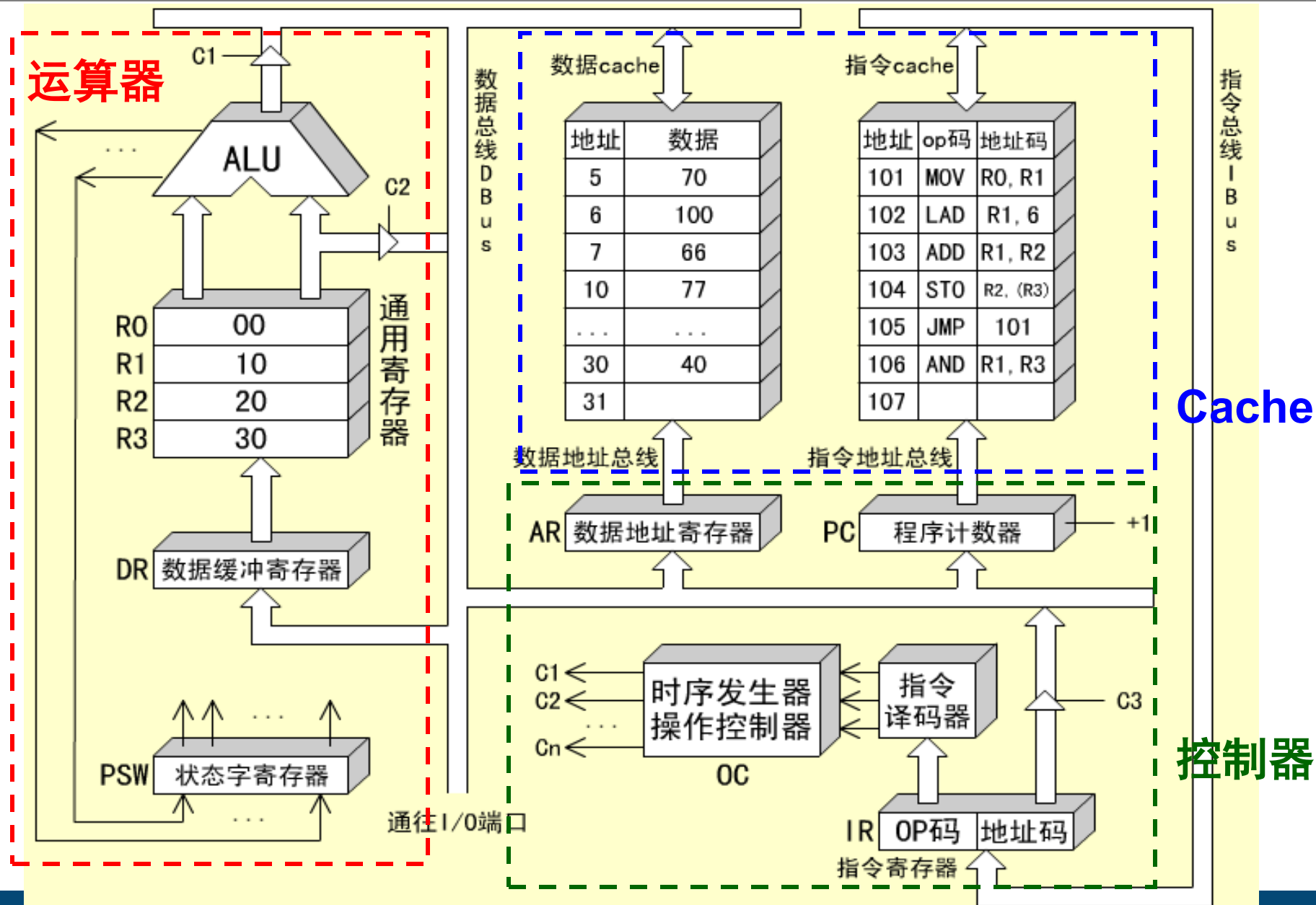
## 6. 指令寄存器（IR）

- 保存当前正在执行的一条指令。
- IR的高位为操作码字段，作为指令译码器的输入。



# >>> CPU模型图

动画演示: CPU基本结构.swf



下列存储器中，汇编语言程序员可见的是（ ）

- ☐ A 存储器地址寄存器（MAR）
- ☒ B 程序计数器（PC）
- ☐ C 存储器数据寄存器（MDR）
- ☐ D 指令寄存器（IR）

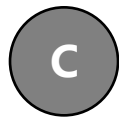
提交

(2018) 减法指令“sub R1,R2,R3”的功能为“(R1) — (R2) → (R3)”,该指令执行后将生成进/借位标志CF和溢出标志OF,。若(R1) = FFFF FFFFH, (R2) = FFFF FFF0H, 则减法指令执行后, CF与OF分别为\_\_\_\_\_。



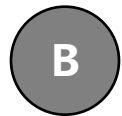
A

CF=0, OF=0



C

CF=0,OF=1



B

CF=1,OF=0

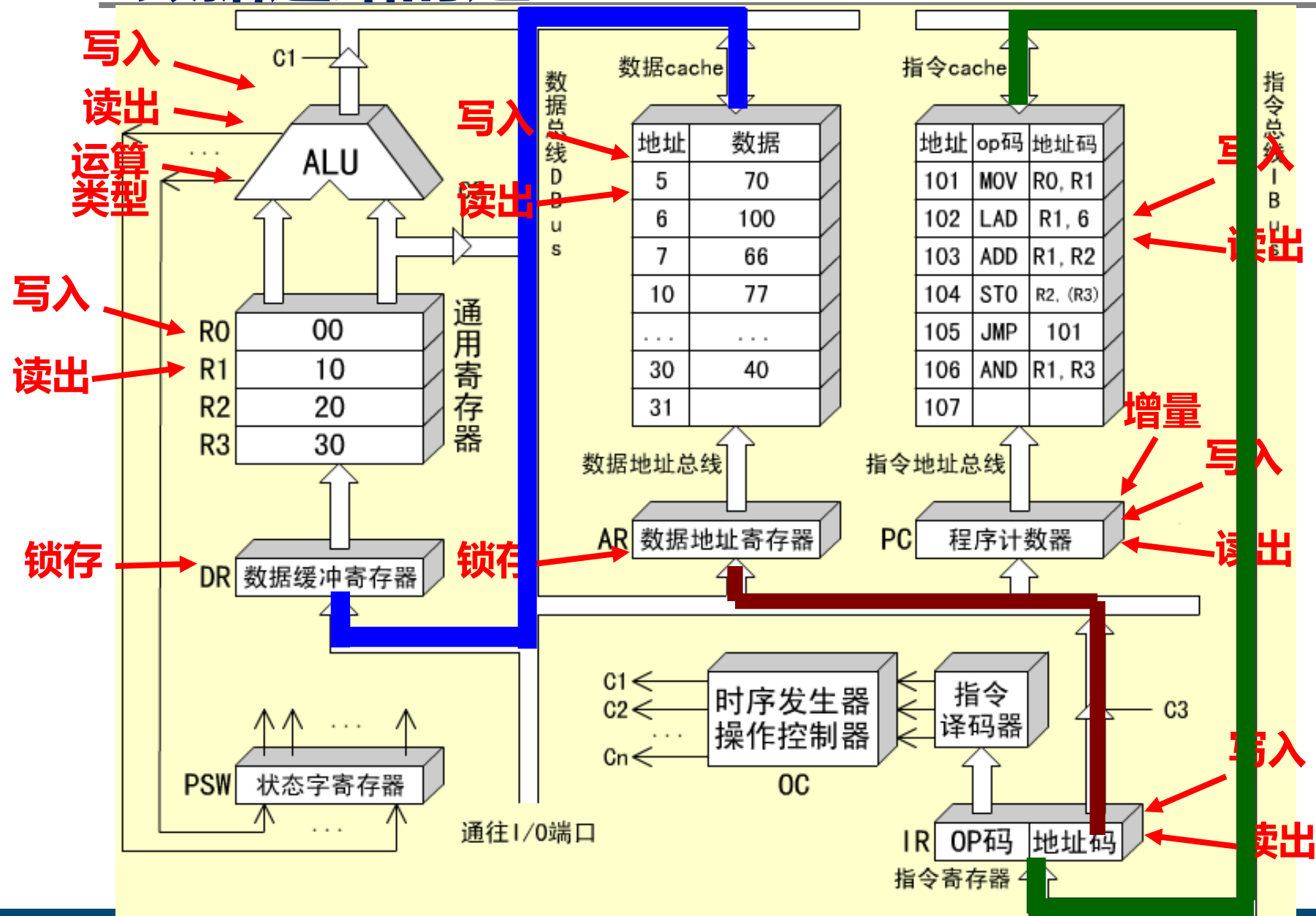


D

CF=1,OF=1

提交

# >>> 数据通路的建立



## 数据通路

- 指令的执行过程就是在CPU内各部件之间以及CPU与外部部件之间，依次、依条件传递信息并进行必要的判断和数据加工的过程
- CPU内部的信息传送通过CPU内部总线进行；CPU与外界交换信息通过系统总线进行

## >>> 5.2 指令周期

---

5.2.1 指令周期的基本概念

5.2.2 MOV R0 , R1指令的指令周期

5.2.3 LAD R1 , 6指令的指令周期

5.2.4 ADD R1 , R2指令的指令周期

5.2.5 STO R2 , (R3)指令的指令周期

5.2.6 JMP 101指令的指令周期

5.2.7 用方框图语言表示指令周期

## >>> 5.2.1 指令周期的基本概念

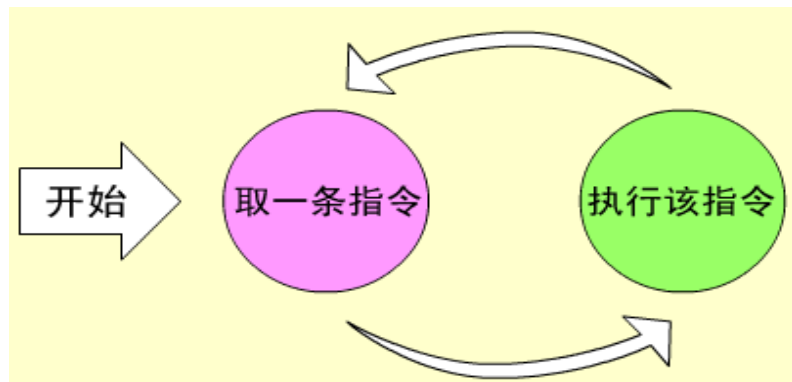
➤ CPU执行程序是一个“取指令—执行指令”的循环过程。

➤ 指令周期

- CPU从内存中**取出**一条指令，并**执行完毕**的时间总和；
- 包含若干个CPU周期

➤ CPU周期（**机器周期**）

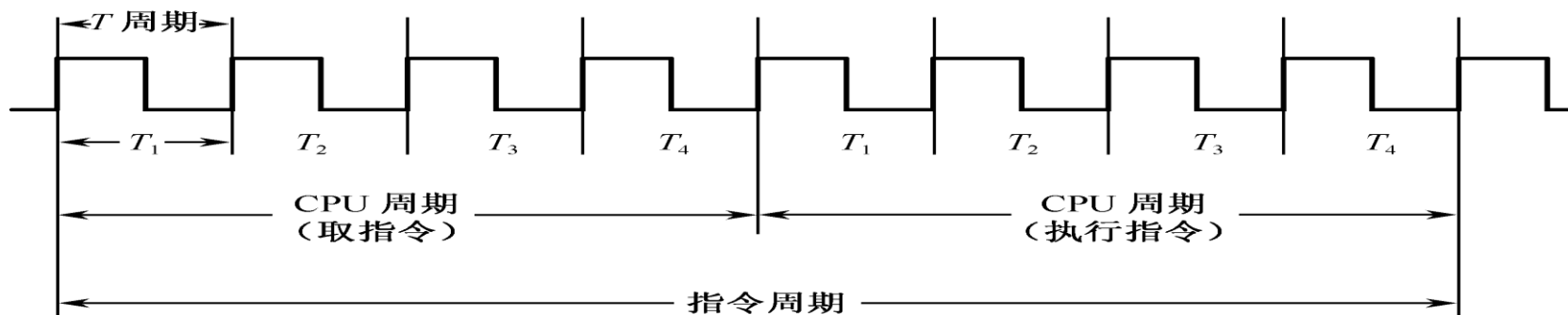
- 指内存中读取一个指令字的最短时间；
- CPU周期通常包含若干个时钟周期
- 时钟周期，又称节拍脉冲或T周期，处理操作的最基本单位。



## >>> 5.2.1 指令周期的基本概念

### ➤ 时钟周期(节拍脉冲、节拍周期或T周期)

- 在一个CPU周期内，要完成若干个微操作。这些微操作有的可以同时执行，有的需要按先后次序串行执行。因而需要把一个CPU周期分为若干个相等的时间段，每一个时间段称为一个节拍脉冲或T周期。
- 时钟周期通常定义为机器主频的倒数。
- 注意：不同的计算机系统定义的术语未必相同，例如：在不采用三级时序的系统中，时钟周期相当于T周期。



# >>> 单周期、多周期和流水线处理器

- **单周期CPU**：在一个时钟周期内完成从指令取出到得到结果的所有工作
  - 所有指令执行时间都以最长时间的指令为准（执行步骤相对固定）
  - 效率低，较少采用。
- **多周期CPU**：把指令的执行分成多个阶段，每个阶段在一个时钟周期内完成
  - 时钟周期短
  - 不同指令所用周期数可以不同
- **流水线处理器**：在多周期CPU设计的基础上，让各个阶段的执行在时间上重叠并行执行
  - 允许功能部件在一条指令执行过程中被重复使用



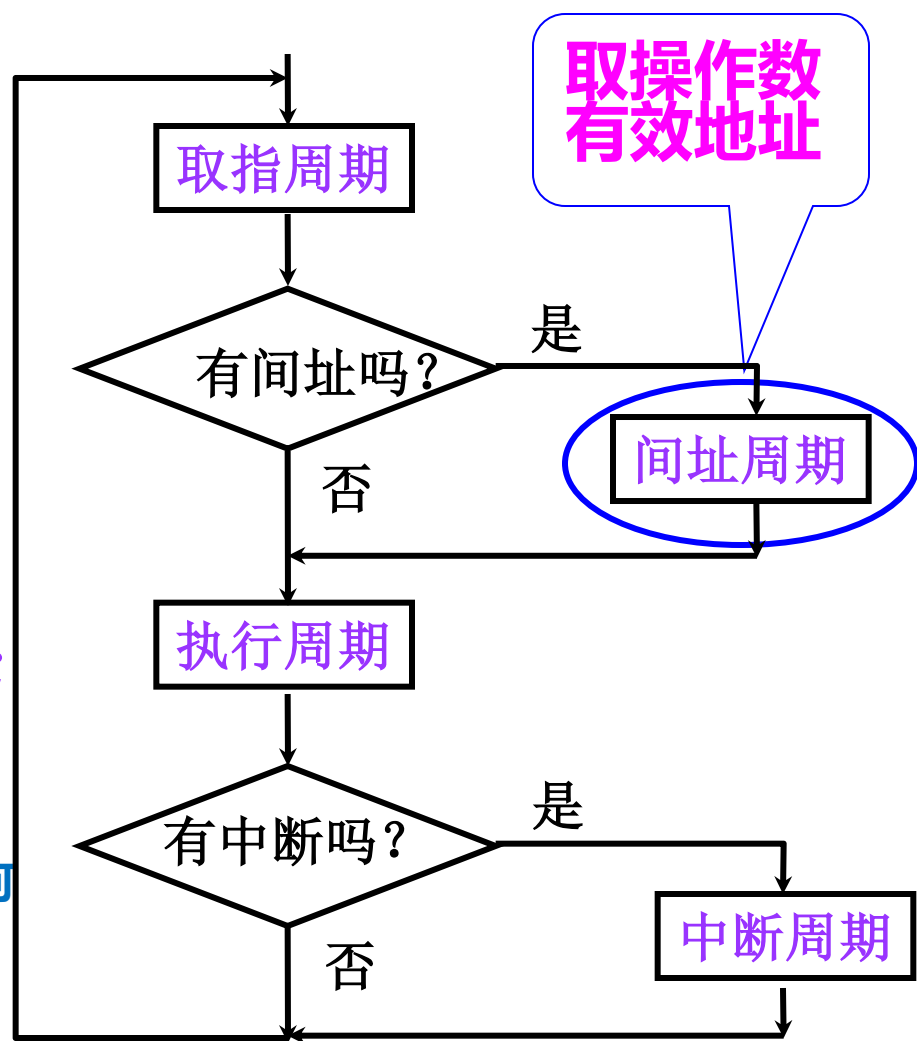
# >>>关于指令周期

➤一个完整的指令周期由若干机器周期：

- 取指周期——间址周期（取操作数阶段）——执行周期——中断周期
- 任何指令周期最少两个机器周期：**取指周期+执行周期**；

➤不同指令的指令周期长度不一定相同。

- 所有指令的**第一个机器周期必为取指周期**；
  - ◆从内存指定单元取出当前指令送至CPU内的指令寄存器IR
  - ◆修改程序计数器PC，以便取下一条指令
  - ◆对指令的操作码字段进行译码测试，以便确定下一步进行何种操作
- 指令的执行周期可包含1~多个机器周期；



# >>>关于指令周期

➤ 一个完整的指令周期由若干机器周期：

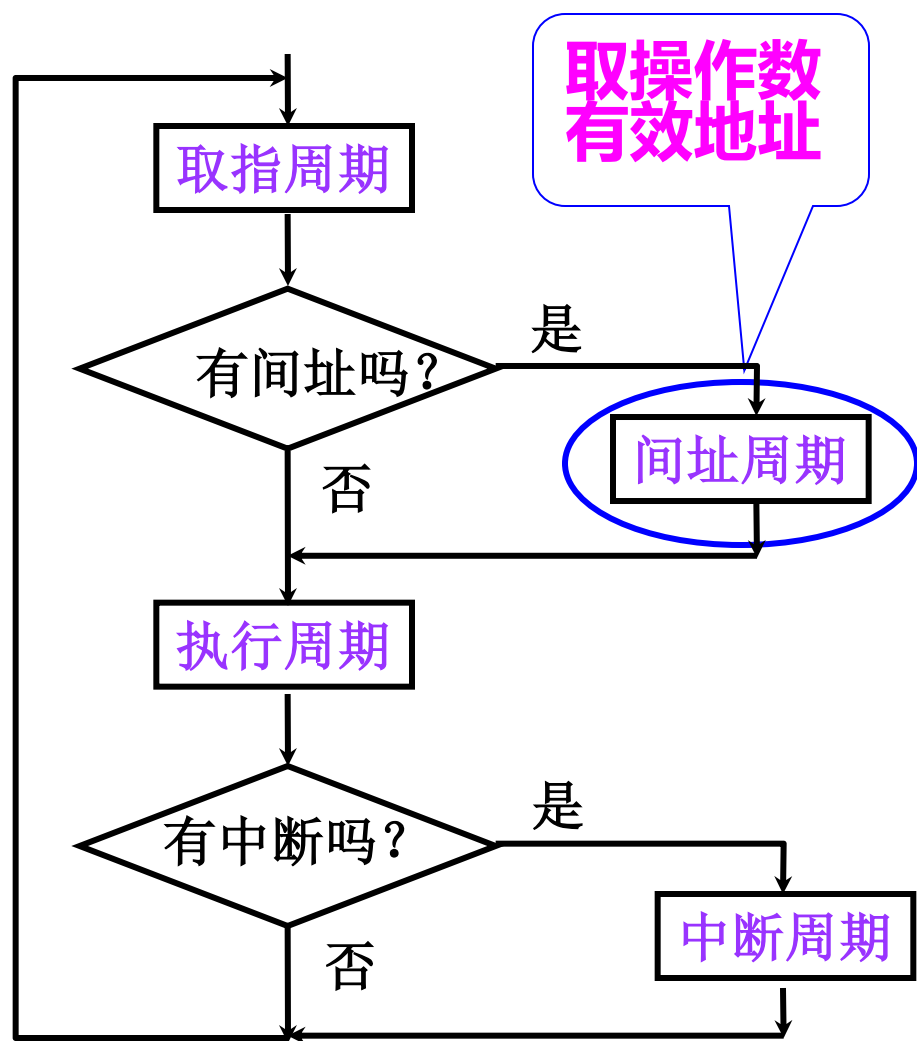
○ **取指周期**——**间址周期**（取操作数阶段）——**执行周期**——**中断周期**

➤ 取操作数阶段

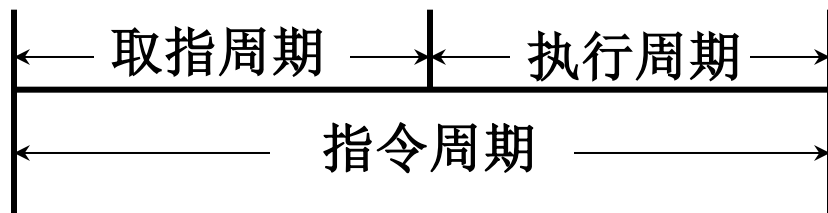
- ◆ 由指令中的形式地址计算实际地址并访问内存取得操作数
- ◆ 只有在需要到内存取操作数的指令中才有该阶段
- ◆ 直接寻址方式和间接寻址方式的取数阶段不相同

➤ 执行周期

- 因各条指令的操作不同而复杂程度各异
- 占用时间相差比较大

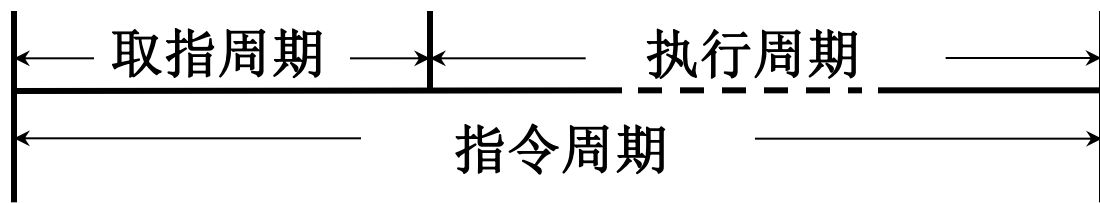


# >>> 不同指令的指令周期不同 (指令的执行过程)



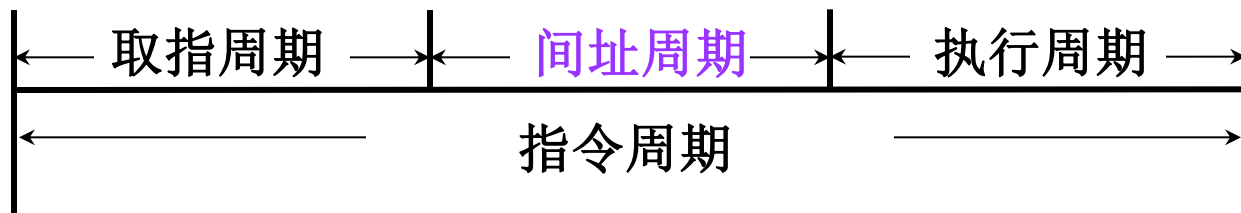
`MOV AX, 0`

不需访存



`ADD AX, BX`

不需访存



`ADD AX, 10[BX]`

需访存

# >>> 一个简单的程序

| 地址  | 数据       |
|-----|----------|
| 5   | 70       |
| 6   | 100      |
| 7   | 66       |
| 10  | 77       |
| ... | ...      |
| 30  | 40 (120) |

| 八进制地址 | 指令            | 说明                                        |
|-------|---------------|-------------------------------------------|
| 100   |               | 程序执行前设置(R0)=00, (R1)=10, (R2)=20, (R3)=30 |
| 101   | MOV R0 , R1   | 传送指令MOV执行(R1)→R0                          |
| 102   | LAD R1 , 6    | 取数指令LAD从6号单元中取数100→R1                     |
| 103   | ADD R1 , R2   | 加法指令ADD执行(R1)+(R2)→R2, 结果为(R2)=120        |
| 104   | STO R2 , (R3) | 存数指令STO用(R3)间接寻址, (R2)=120写入30号单元         |
| 105   | JMP 101       | 转移指令JMP改变程序执行顺序, 转到101号单元                 |
| 106   | AND R1 , R3   | 逻辑与指令AND执行(R1) · (R3) →R3                 |

## >>> 5.2.2 MOV R0, R1指令的指令周期

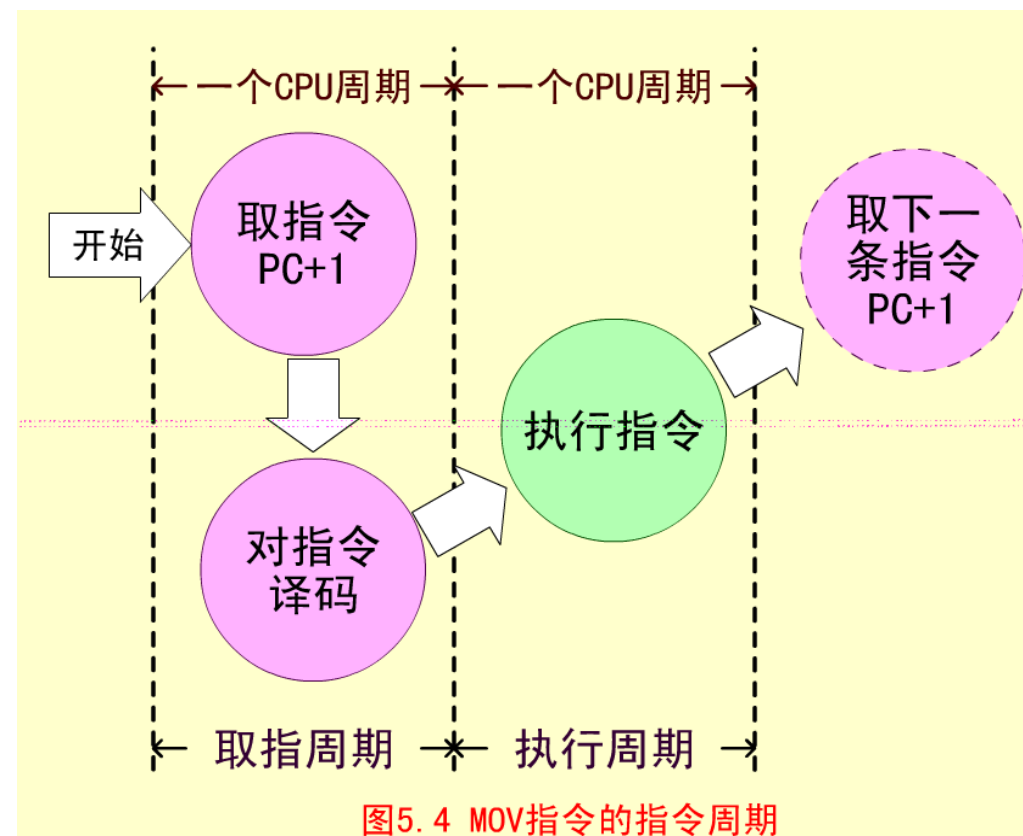
➤ **MOV** 是一条 **RR** 型指令，它需要两个 **CPU** 周期：（不需访存，**RR** 型指令）

### ○ 取指周期

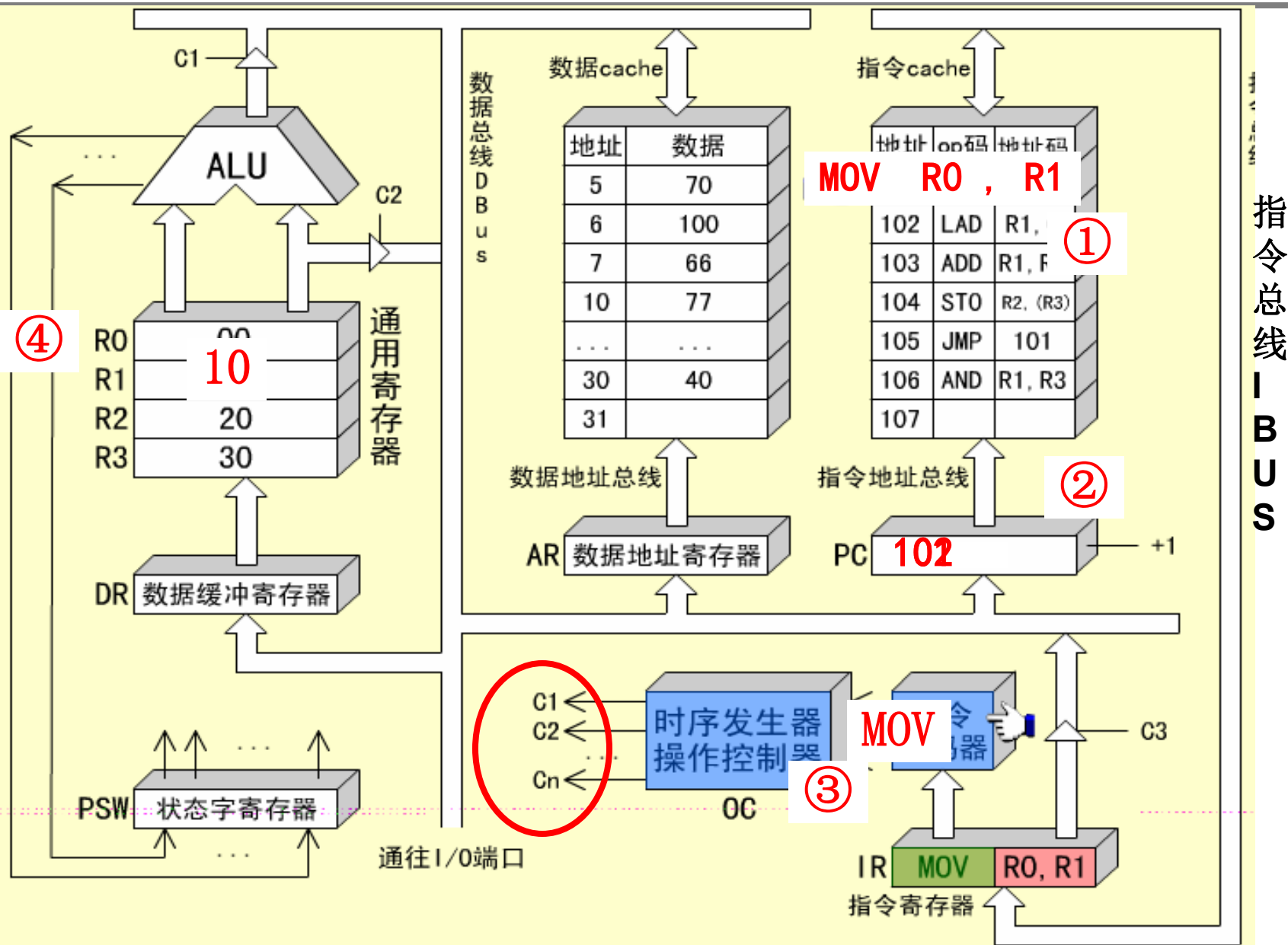
- ① 从存储器中取出指令；
- ② 程序计数器PC加1；
- ③ 译码或测试指令操作码，发出控制信号；

### ○ 执行周期

- ◆ 在控制信号的作用下，将R1中的数据经过ALU送入R0；



# >>> MOV R0, R1指令的执行过程演示



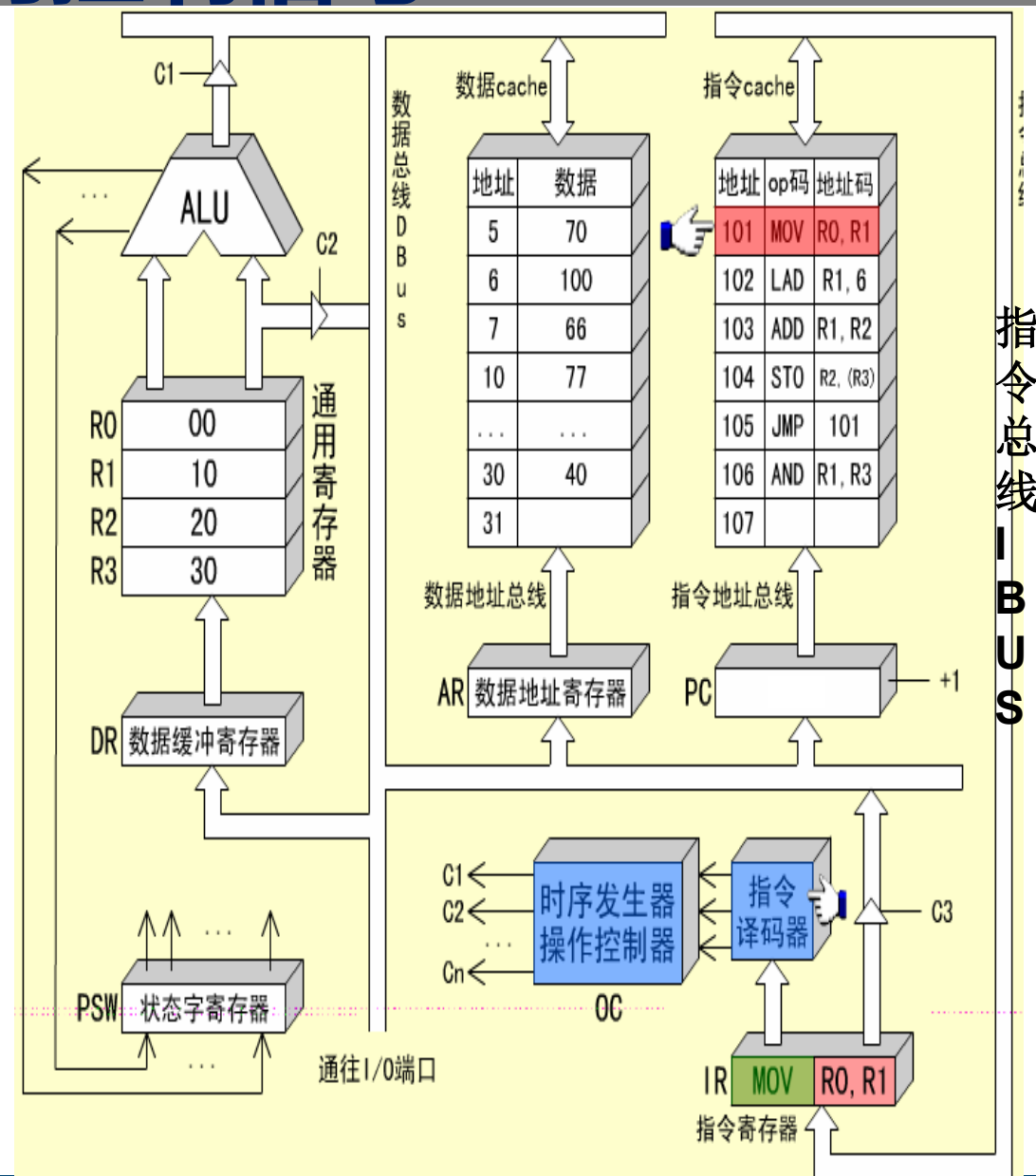
# >>> MOV R0, R1指令周期中的控制信号

## 1. 取指周期

- ①  $PC \rightarrow \text{ABUS} \rightarrow \text{指令Cache}$ ，译码并启动；
- ② 指令Cache  $\rightarrow$  IR；
- ③  $PC+1 \rightarrow PC$ ，为取下条指令做好准备；PC增量
- ④ IR中的操作码被译码或测试，CPU识别出是指令MOV。

## 2. 执行指令周期

- ①  $R1 \rightarrow \text{ALU}$ ，R1中数据通过ALU传送；
- ②  $\text{ALU} \rightarrow \text{DBUS} \rightarrow \text{DR} \rightarrow R0$ ；



## >>> 5.2.3 LAD R1, 6指令的指令周期

➤ **功能：**将内存给定单元的数据送至通用寄存器

**LAD R1, 6:** (6) → R1 内存直接寻址

➤ **LAD指令是RS型指令，需要访存获取操作数，共包含三个CPU周期：**

① **取指周期**

② **执行周期 (2个CPU周期)**

a) **使用直接地址访问数据Cache，获得源操作数；**

b) **将源操作数送入通用寄存器R1；**

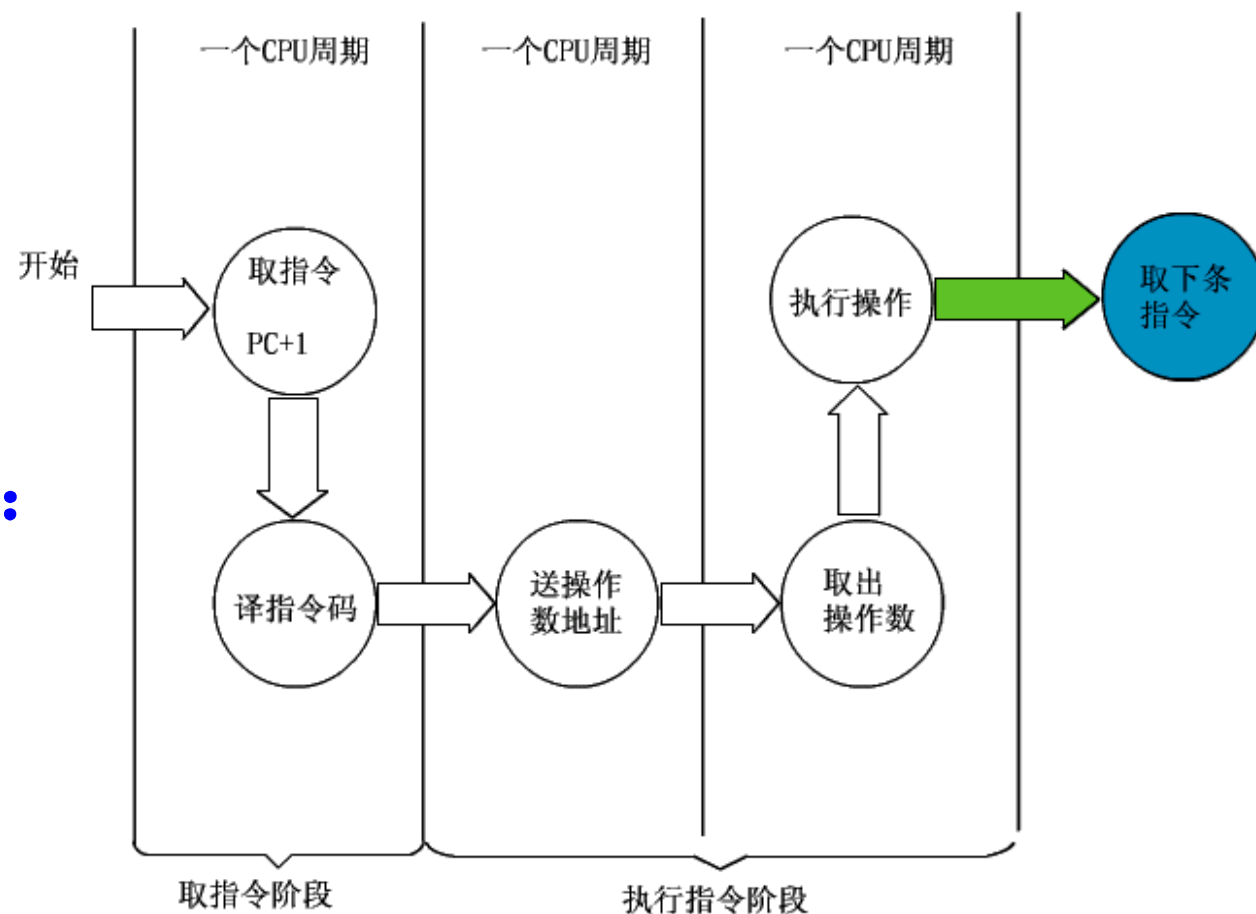
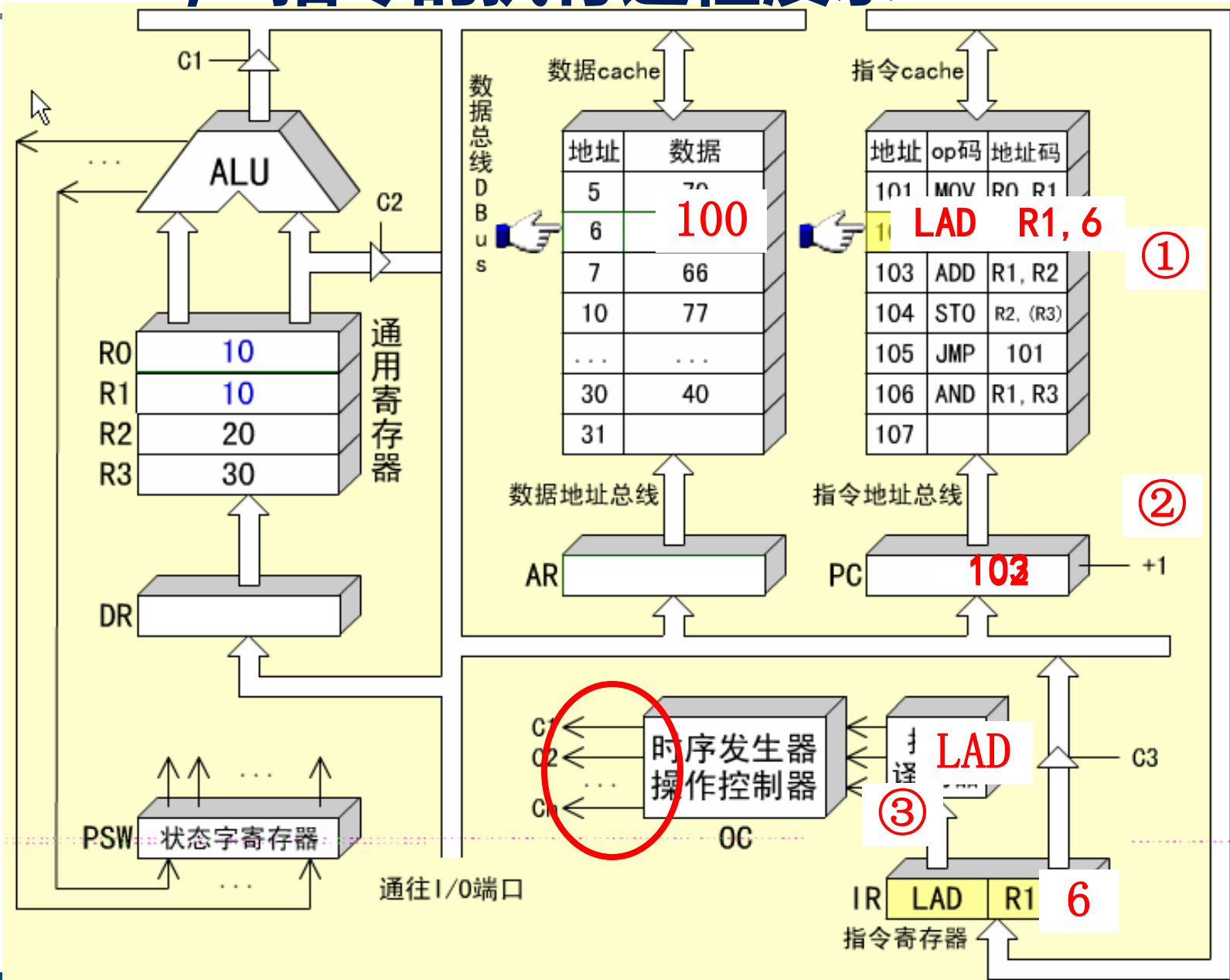


图5.7 直接访内指令的指令周期



# >>> LAD R1, 6指令的执行过程演示



# >>> LAD R1, 6指令周期中的控制信号

## 1.LAD取指周期

○与MOV指令的取指周期一样。

## 2.LAD指令的执行周期

○IR → DBUS → AR;

IR读  
AR锁存

◆ AR → ABUS → 数据Cache, 译码并启动;

○数据Cache → DBUS → DR → R1;

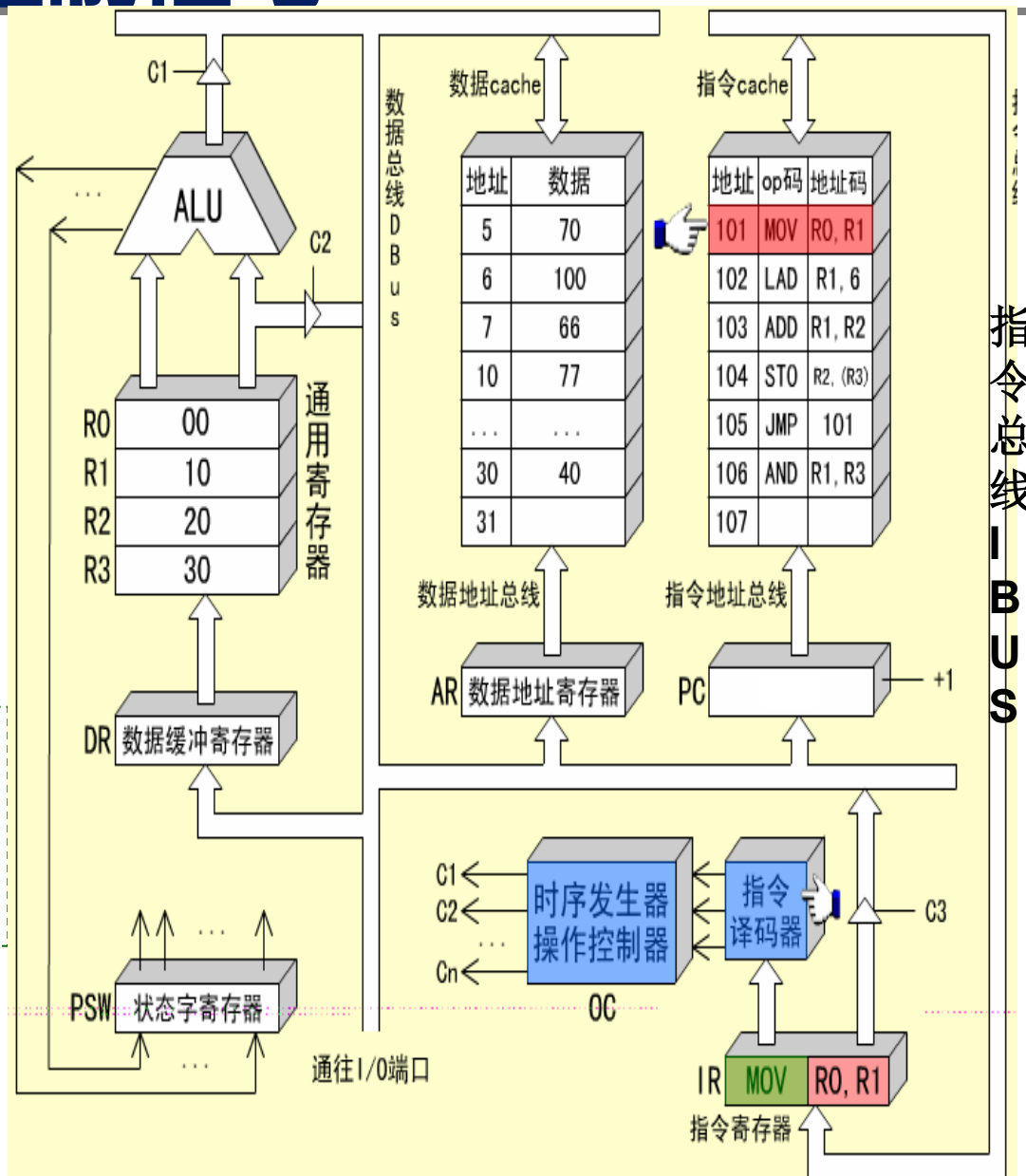
数据Cache启动

数据Cache读  
DR锁存  
R1写

执行周期需要两个CPU周期的原因是:

源操作数的地址和源操作数都需要经过

DBUS传送;



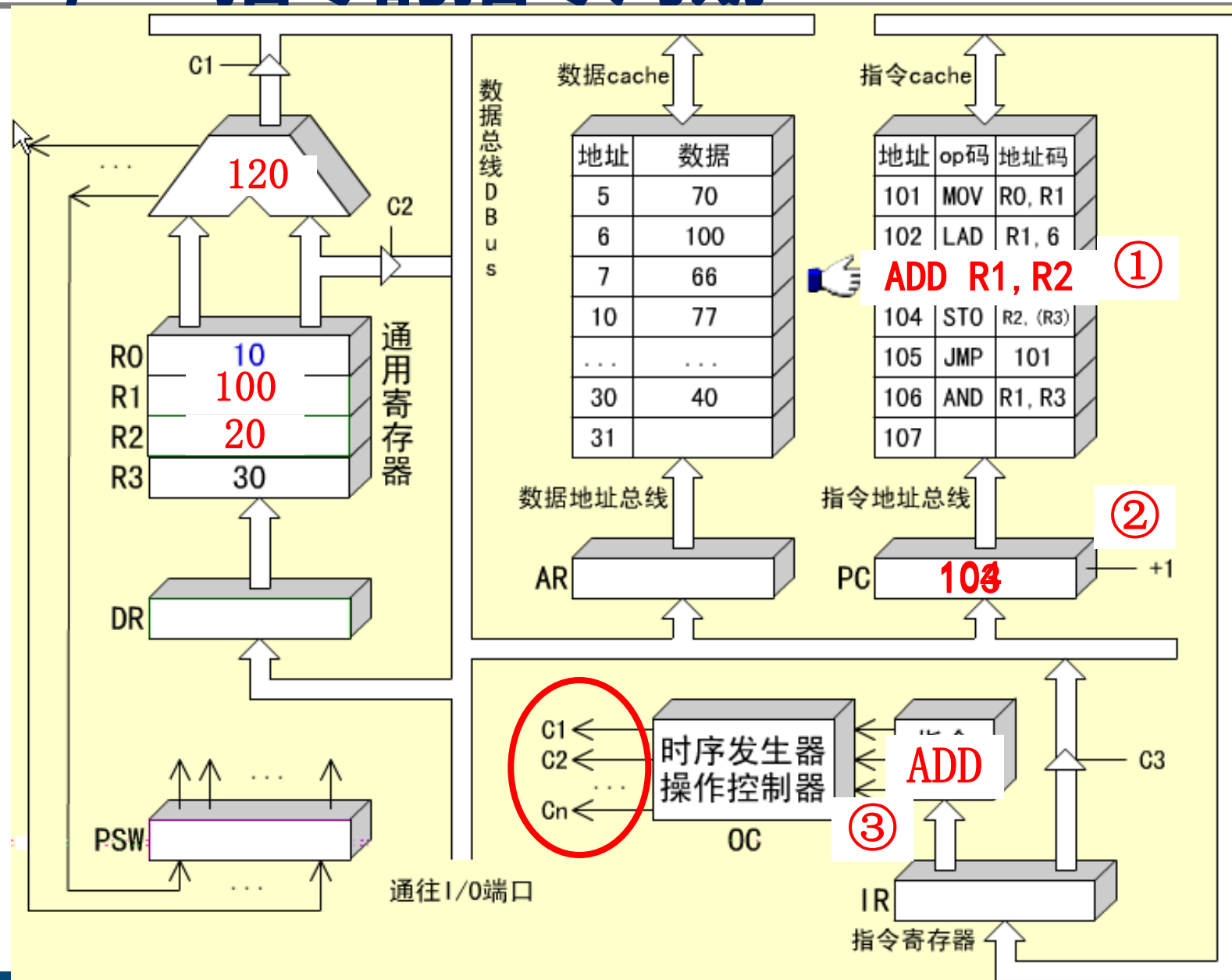
## >>> 5.2.4 ADD R1, R2指令的指令周期

➤ **ADD指令的指令周期由两个CPU周期组成。**

① **取指周期（略）**

② **执行周期**

- ◆ **从寄存器R1、R2中取出数据，作为源操作数；**
- ◆ **将两数据送往ALU，并使ALU进行加运算；**
- ◆ **结果保存到R2中。**



# >>> ADD R1, R2指令周期中的控制信号

## ①取指周期（略）

○ 与MOV指令的取指周期一样。

## ②执行周期

○ R1、R2→ALU;

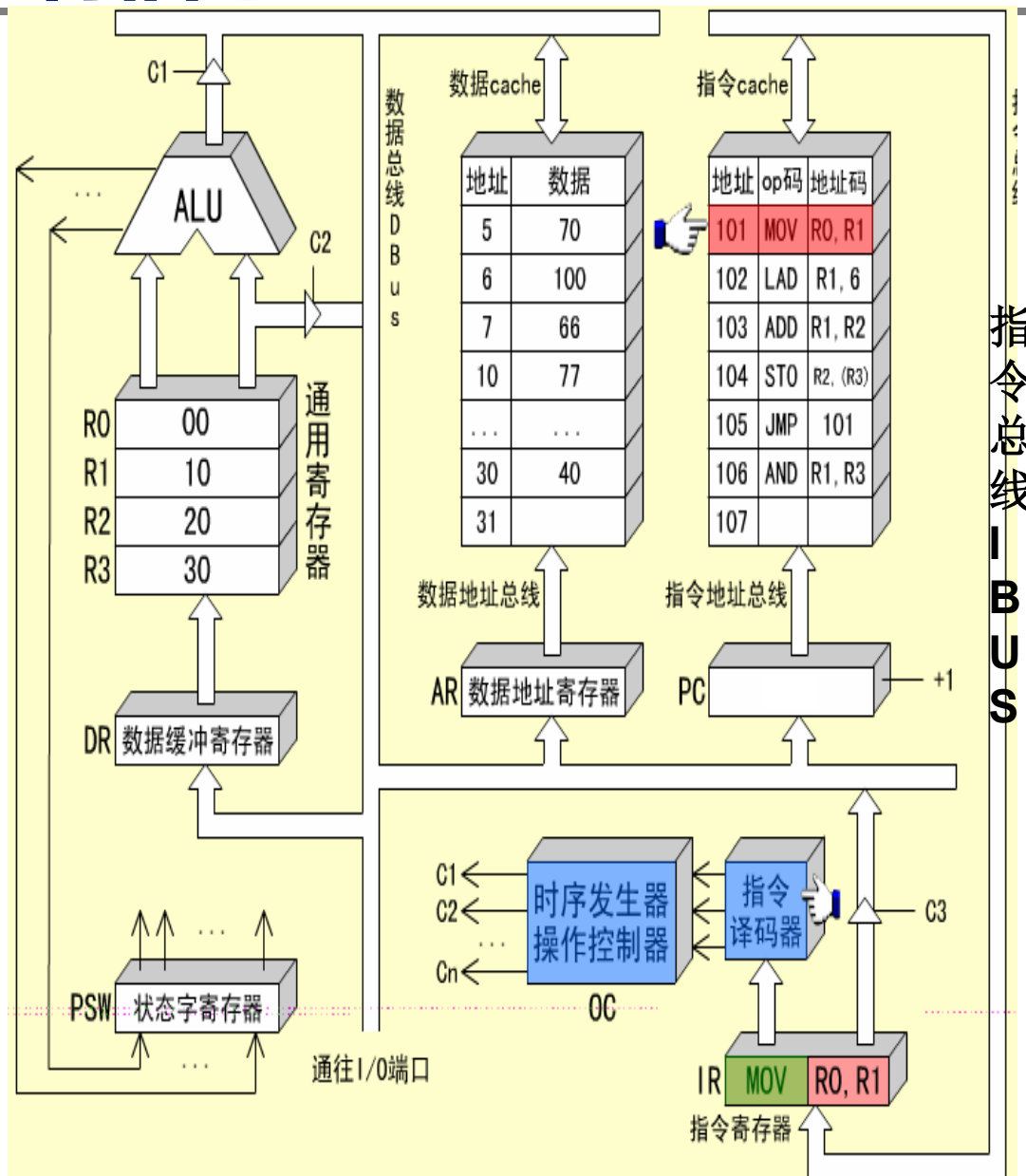
○ ALU做加运算，将两数相加;

○ ALU→DBUS→DR→R2, 保存结果;

ALU加

ALU输出  
DR锁存  
R2写

R1读  
R2读



## >>> 5.2.5 STO R2, (R3)指令的指令周期

➤ **功能：**将寄存器内容送入由寄存器指定的内存单元。STO R2, (R3): R2→(R3)

➤ **STO指令是RS型指令，需要3个CPU周期。**

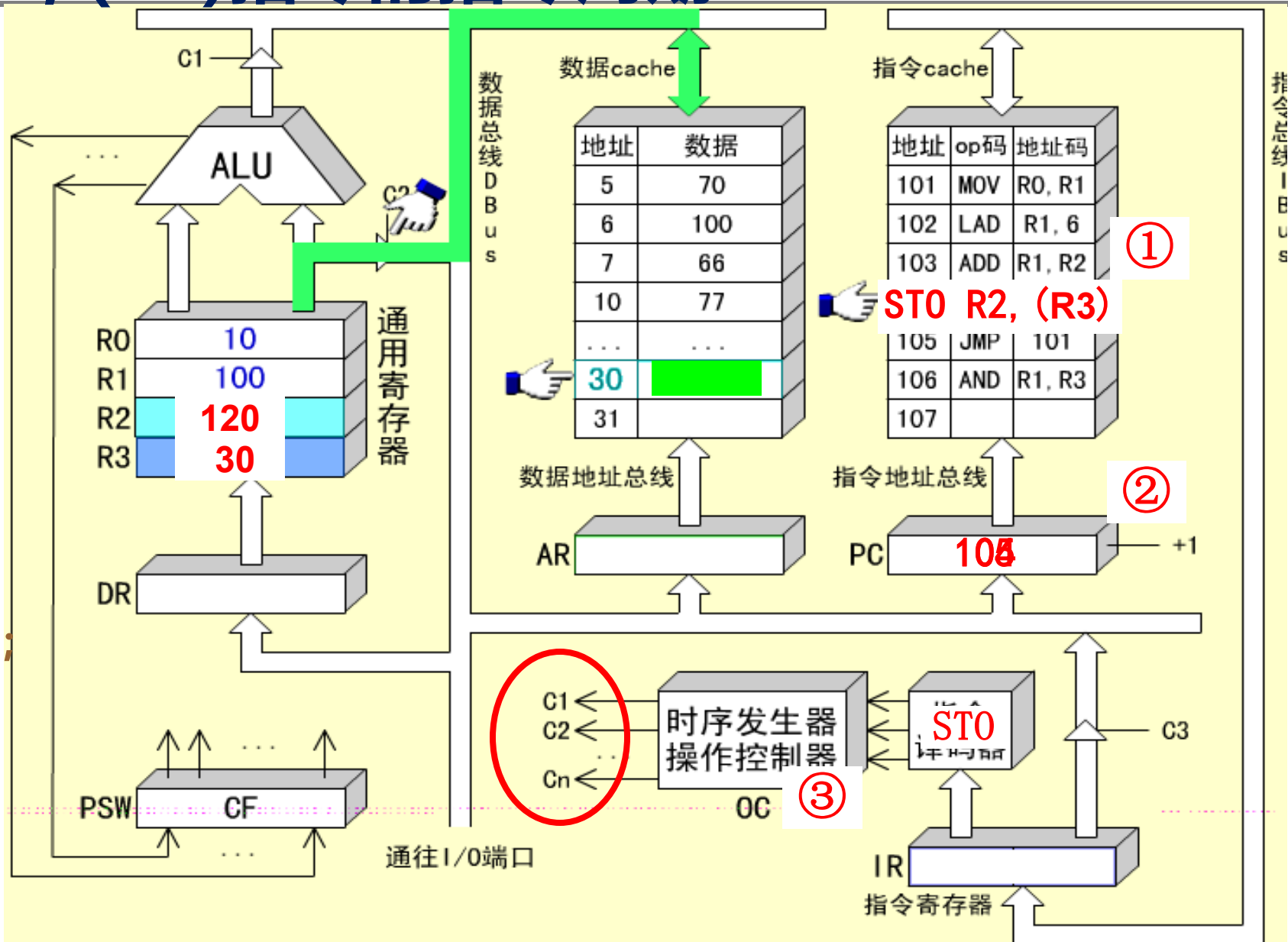
① **取指周期（略）**

② **间址周期**

◆ 根据R3中的地址寻址存储单元；

③ **执行周期**

◆ 将寄存器R2中的数据送入选定的存储单元；

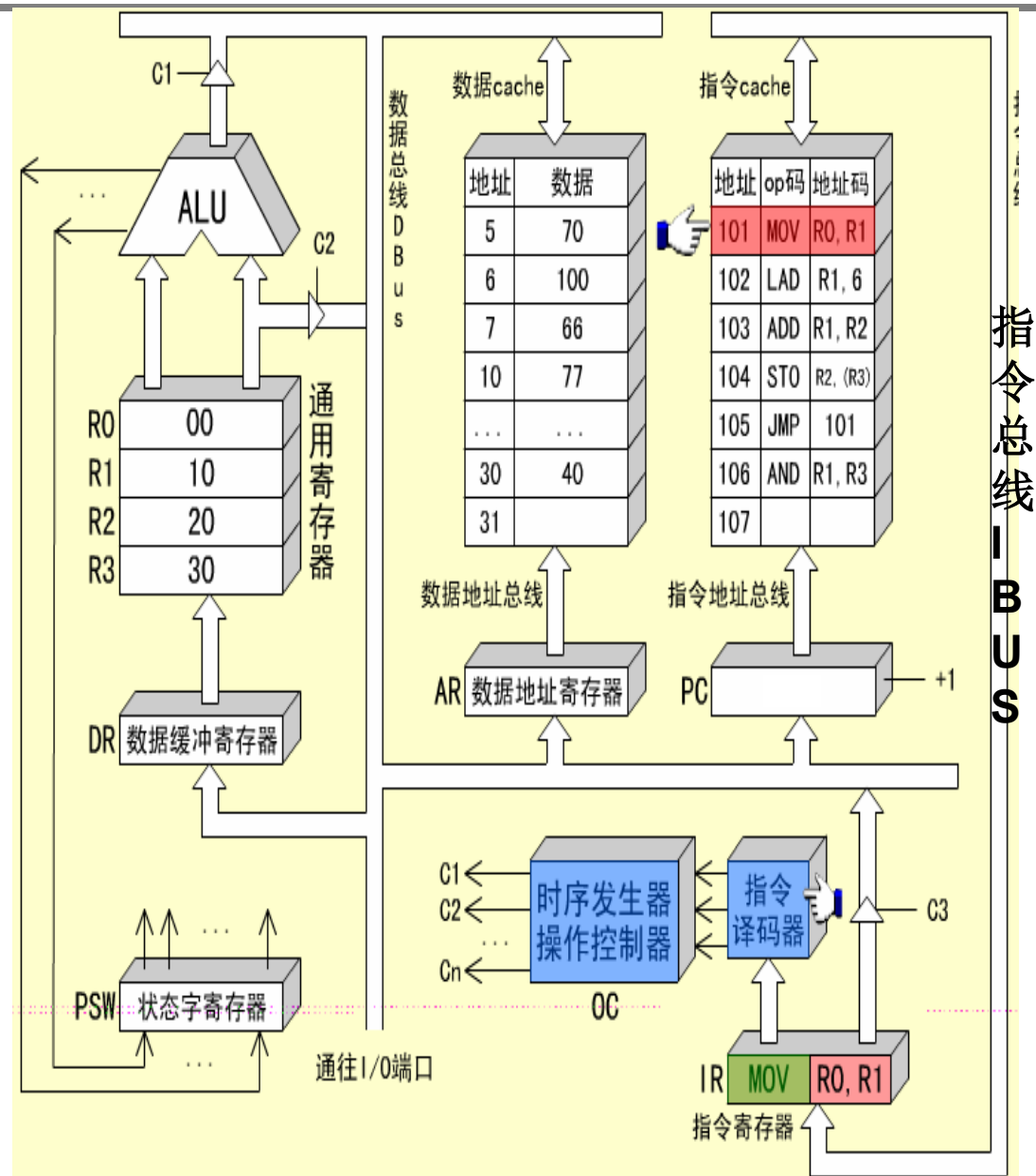


# >>> STO R2, (R3)指令周期中的控制信号

## 1. 取指周期（略）

## 2. 执行周期

- **R3→DBUS→AR**, 发出地址启动数据Cache; **R3读AR锁存**
  - ◆ 该过程为间址周期;
- **R2→DBUS→数据Cache**; **R2读数据Cache写**



## >>> 5.2.6 JMP 101指令的指令周期

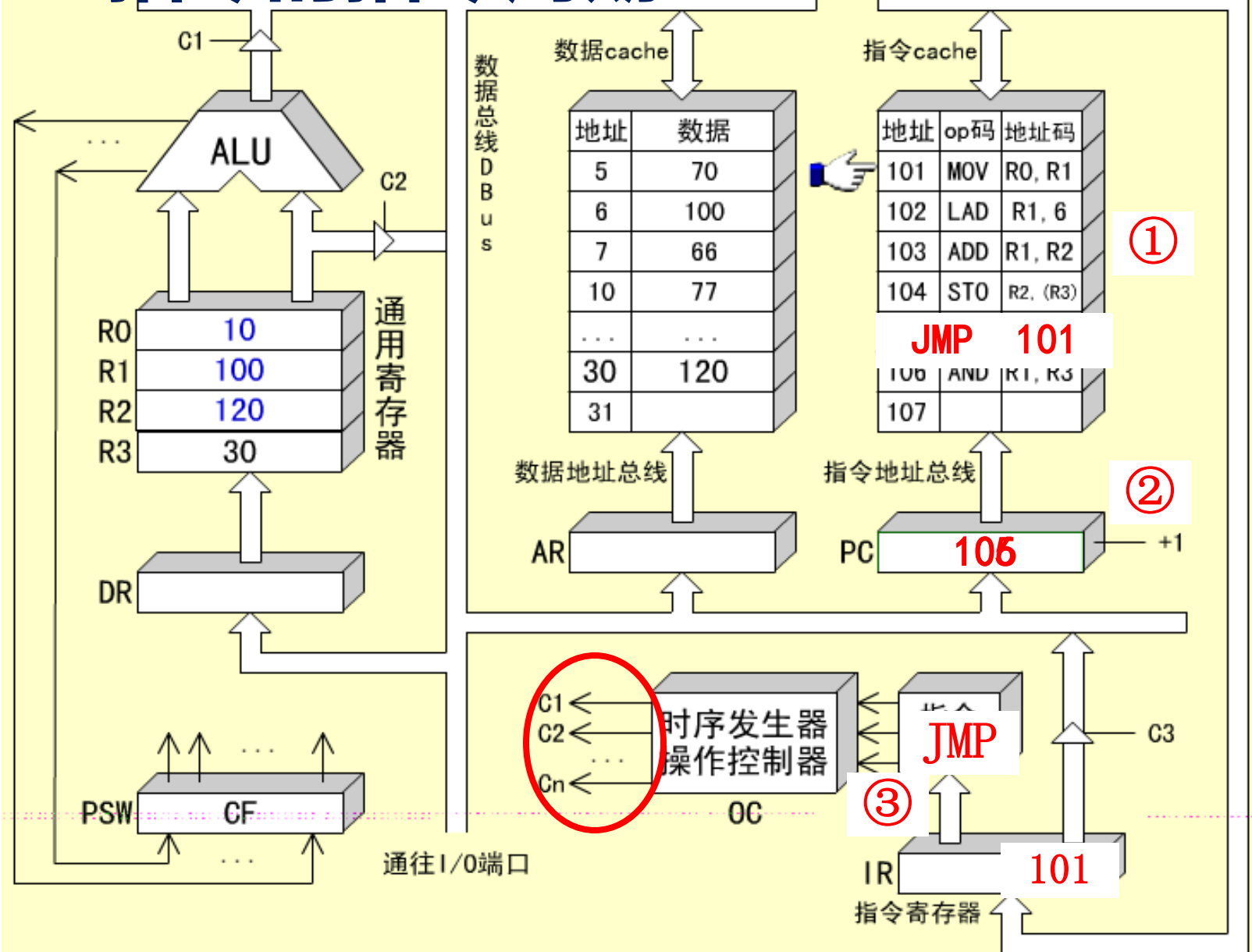
JMP 101指令的执行过程演示

- **JMP**指令是一条无条件转移指令，用来改变程序的执行顺序；
- **JMP**指令的执行需要两个**CPU**周期：

①取指周期（略）

②执行周期

- ◆ 使用**JMP**指令中的直接地址为PC赋值；





# >>> JMP 101指令周期中的控制信号

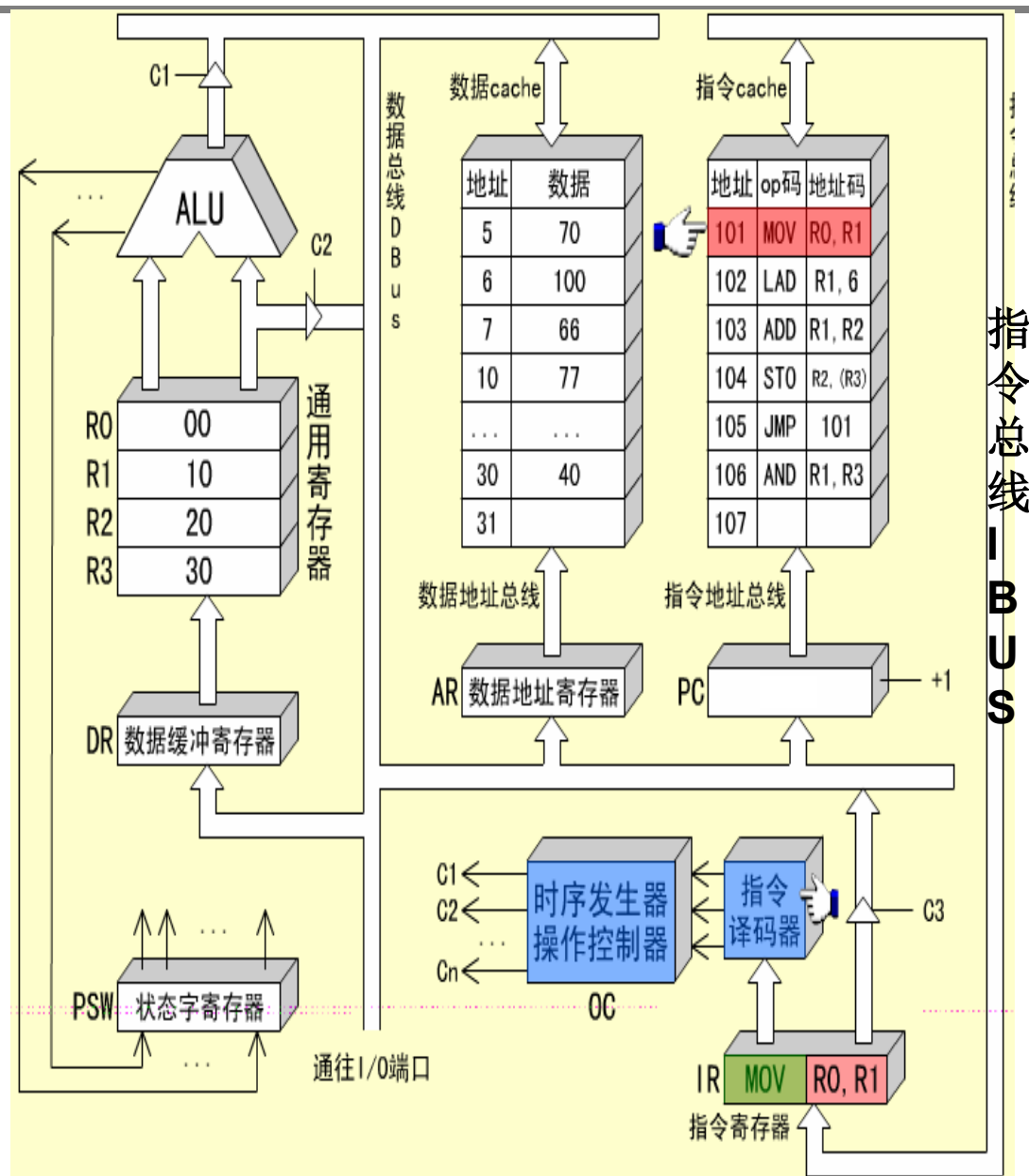
1. 取指周期（略）

2. 执行周期

○ IR→DBUS→PC;

IR读  
PC写

5条指令执行过程的动画演示





## >>> 5.2.7 用方框图语言表示指令周期

➤在进行中央处理器设计时可以用方框语言来描述一条指令的指令周期

➤方框

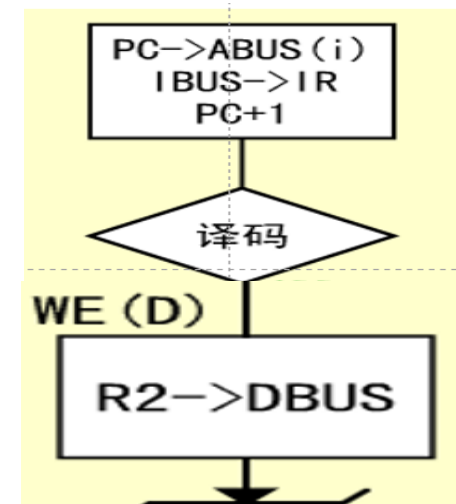
- 一个方框代表一个CPU周期;
- 方框中的内容表示数据通路的操作或某种控制操作。

➤菱形

- 通常用来表示某种判别或测试;
- 时间上依附于之前一个方框的CPU周期，而不单独占用一个CPU周期;

➤~（公操作符号）

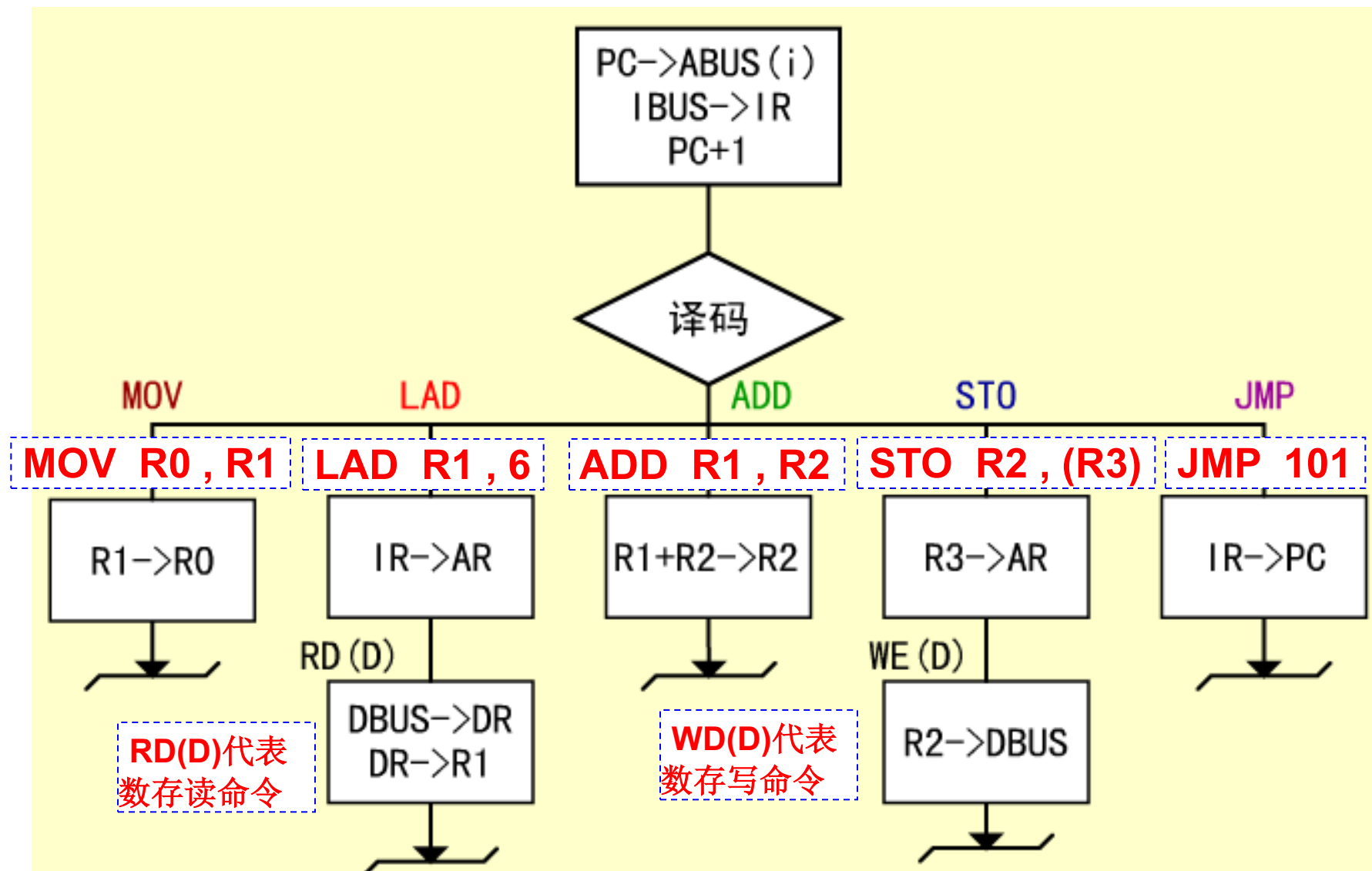
- 表示一条指令已经执行完毕，转入公操作。
- 所谓公操作就是一条指令执行完毕后，CPU所开始的一些操作，比如对外围设备请求的处理等。



动画演示:

[方框图语言描述指令周期.swf](#)

## >>>P162 图5.15方框图语言表示的指令周期

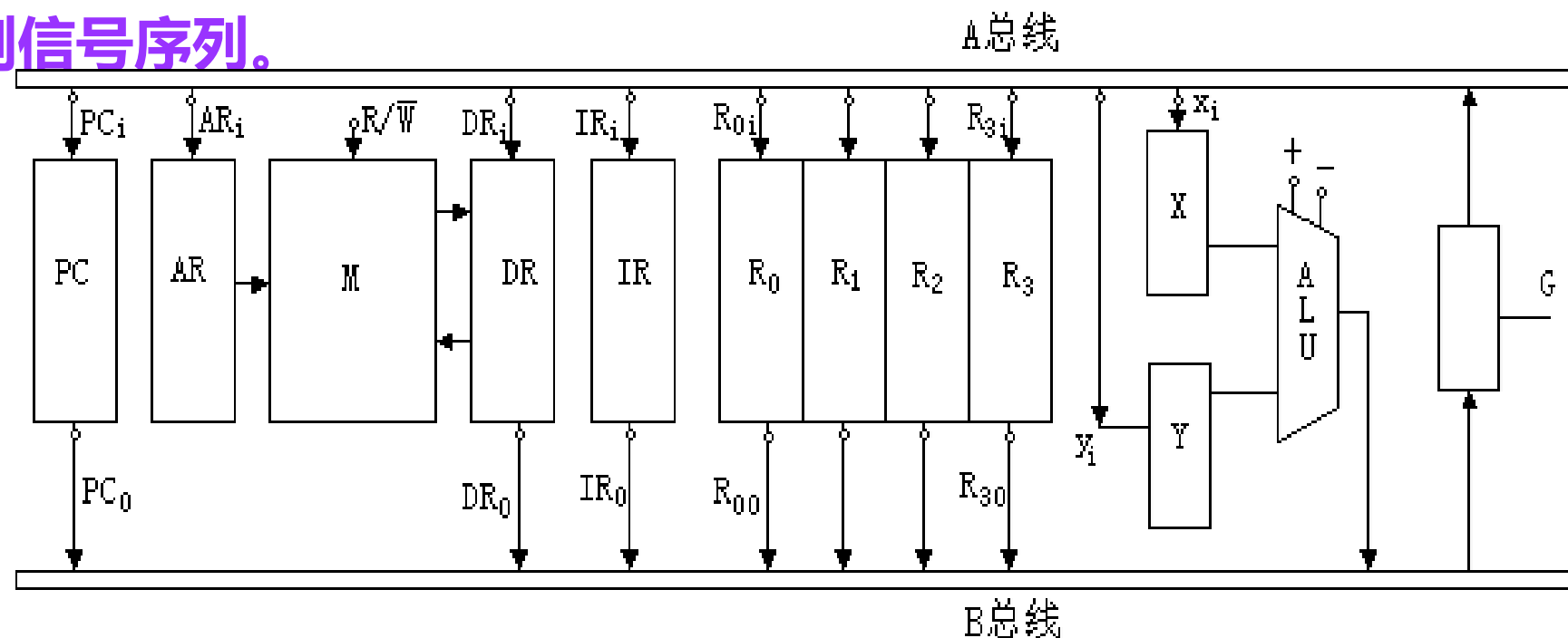


## >>> 课本P161 【例5.1】

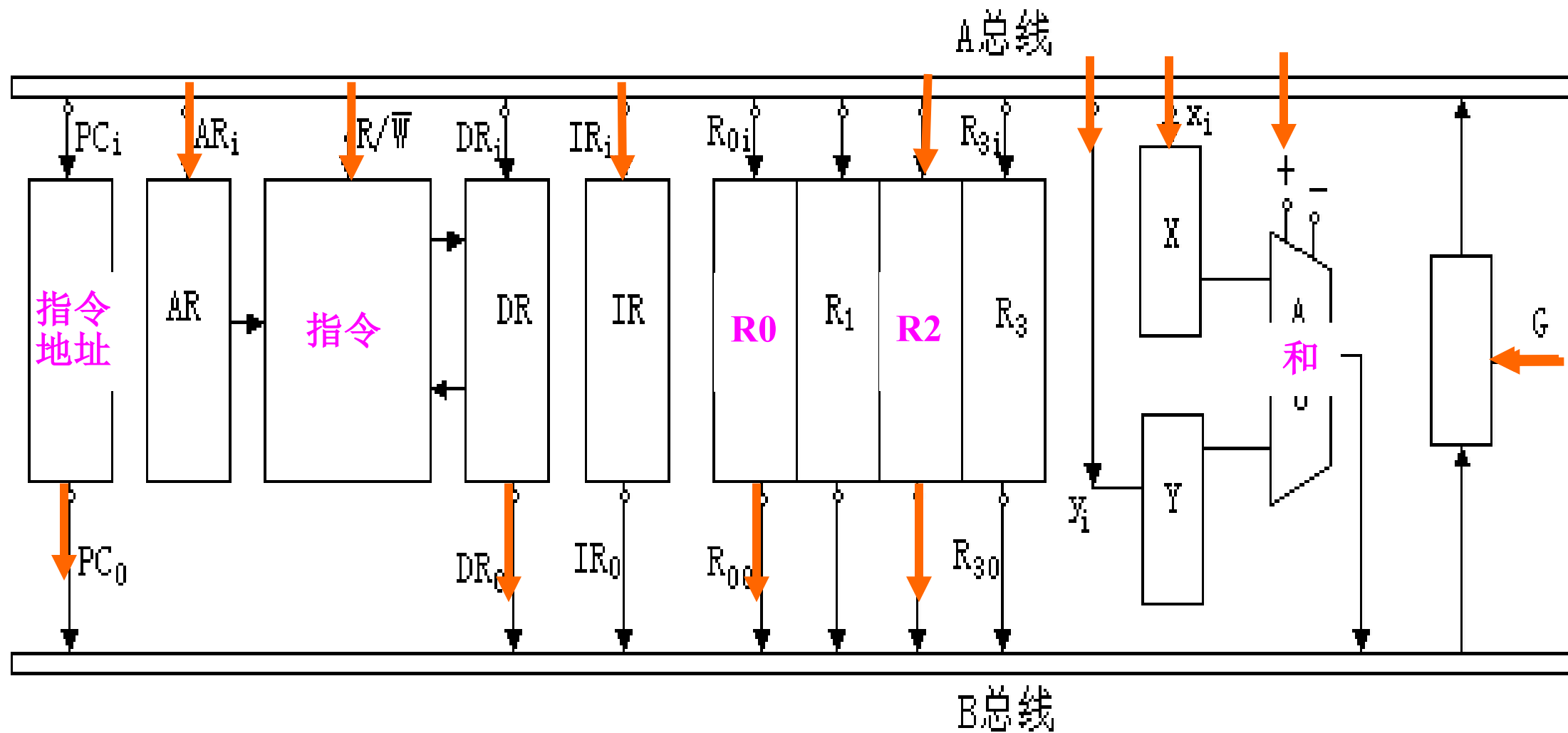
➤ 图5.15所示为双总线结构机器的数据通路，各构成部件如图，线上标注有小圈表示有控制信号，未标字符的线为直通线。

- ① “ADD R2, R0”指令完成 $(R0)+(R2) \rightarrow R0$ 的功能操作，画出其指令周期流程图，并列出相应的微操作控制信号序列。
- ② “SUB R1, R3”指令完成 $(R3)-(R1) \rightarrow R3$ 的操作，画出其指令期流程图，并列出相应的微操作控制信号序列。

IR为指令寄存器  
PC为程序计数器（具有自增功能）  
M为主存（受R/W信号控制）  
AR为地址寄存器  
DR为数据缓冲寄存器  
R0~R3为寄存器组  
ALU由加、减控制信号决定完成何种操作  
控制信号G控制的是总线旁路器

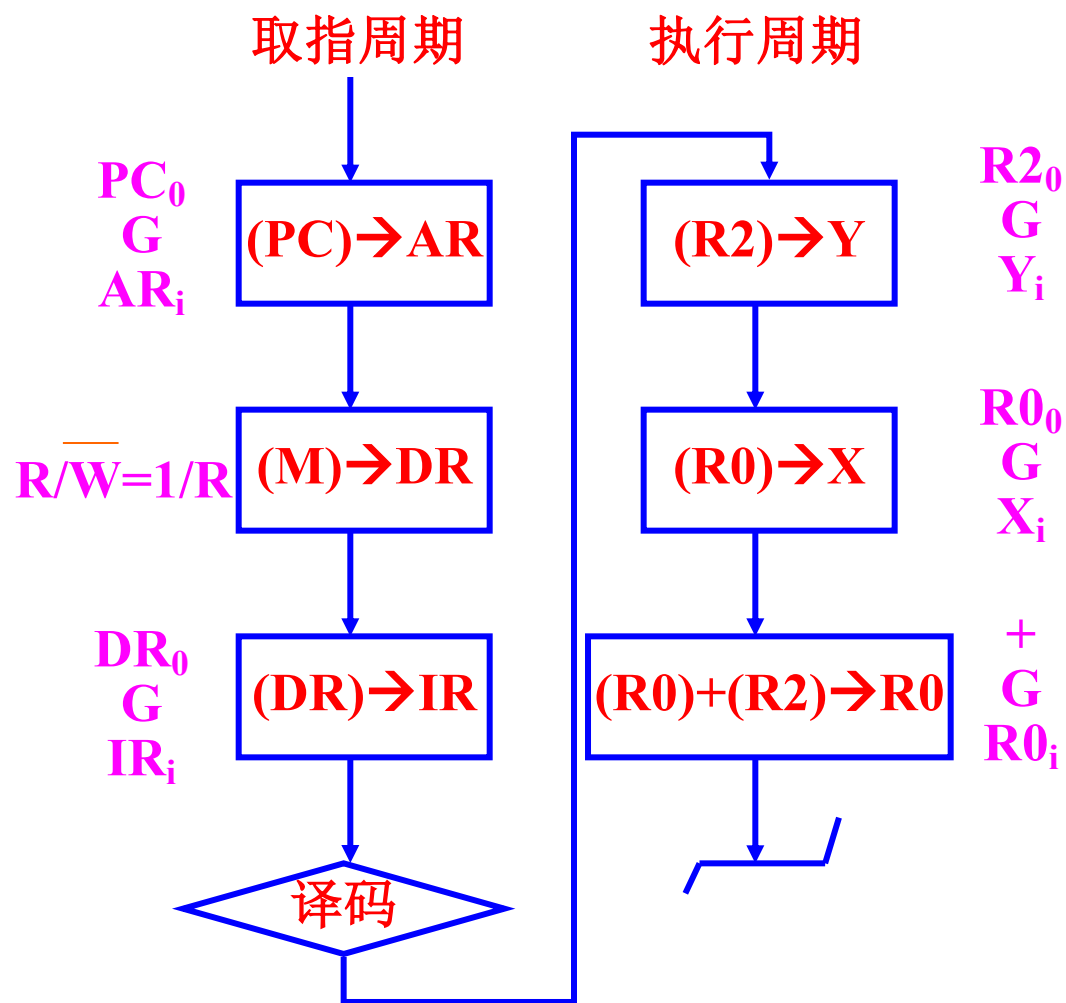


# >>> (1)分析指令ADD R2, R0的执行过程



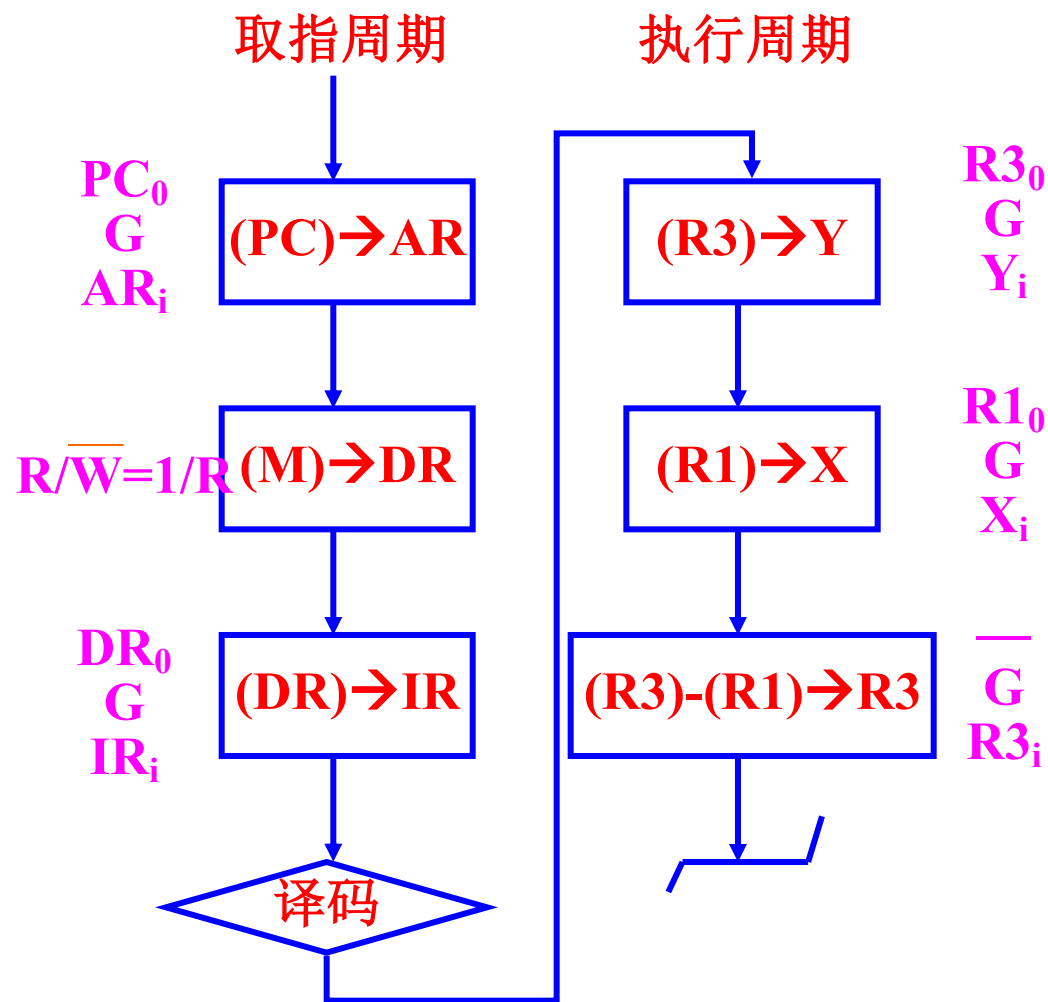
# >>> (1) “ADD R2, R0” → (R0) + (R2) → R0

- 指令周期应包括取指周期和执行周期；
- 执行周期中应首先将R0、R2两寄存器的内容送入ALU的操作数缓冲器中，再执行加法运算；



## >>> (2) “SUB R1, R3” → (R3) - (R1) → R3

- 指令周期应包括取指周期和执行周期；
- 取指周期与ADD指令完全相同；
- 执行周期与ADD指令不同之处在于ALU的控制信号为“—”



## >>> 5.4 微程序控制器

---

5.4.1 微程序控制原理

5.4.2 微程序设计技术

## >>> 5.4.1 微程序控制原理

### ➤ 基本思想

- 仿照通常的解题程序的方法，把操作控制信号编成所谓的“微指令”，存放到一个只读存储器里；
  - 当机器运行时，一条一条地读出这些微指令，从而产生全机所需要的各种操作控制信号，使部件执行规定的操作。
- 由上述可知微程序设计技术是利用软件方法来设计硬件的一门技术。
- 具有规整性、灵活性、可维护性等一系列优点；
  - 逐渐取代了早期的组合逻辑控制器，已被广泛地应用。
- 要解决的问题：如何将一条机器指令编辑成微指令集合或微操作集合（多个微操作可能对应一条微指令）



# >>> 举例说明微程序控制思想

- 通过对右表中的指令分析，可得以下CPU周期的控制信号

① 取指：PC读、指令Cache读、IR写、PC增量

② 间址：IR读、AR锁存

③ 间址：R3读、AR锁存

④ 执行：R1读、ALU传送控制、ALU输出、DR锁存、R0写

⑤ 执行：数据Cache读、DR锁存、R1写

⑥ 执行：R1读、R2读、ALU加、ALU输出、DR锁存、R2写

⑦ 执行：R2读、数据Cache写

⑧ 执行：IR读、PC写

- 指令LAD R1 , (R3)可经历①③⑤步骤完成；

- 指令STO R2 , 6可经历①②⑦步骤完成。

| 地址  | 指令            |
|-----|---------------|
| ①④  | MOV R0 , R1   |
| ①②⑤ | LAD R1 , 6    |
| ①⑥  | ADD R1 , R2   |
| ①③⑦ | STO R2 , (R3) |
| ①⑧  | JMP 101       |

# >>> 1、微命令和微操作

- 控制部件与执行部件之间存在两种联系
  - 控制部件向执行部件发出控制信号（各种控制命令—微命令）；
  - 执行部件向控制部件返回状态信息；
- 微命令
  - 控制部件通过控制线向执行部件发出的各种控制命令；它是构成控制序列的最小单位
  - 微命令是控制计算机各部件完成某个基本微操作的命令。
    - ◆ 例如：打开或关闭某个控制门的电位信号、某个寄存器的打入脉冲等。

控制信号线

反馈信息

# >>> 1、微命令和微操作

➤ 微操作：是微命令的操作过程。

执行部件中的最基本的操作

- 执行部件接受微命令后所执行的操作；
- 微命令和微操作是一一对应的。
- 微操作是执行部件中最基本的操作。
- 微命令是微操作的控制信号，微操作是微命令的操作过程。

➤ 状态测试

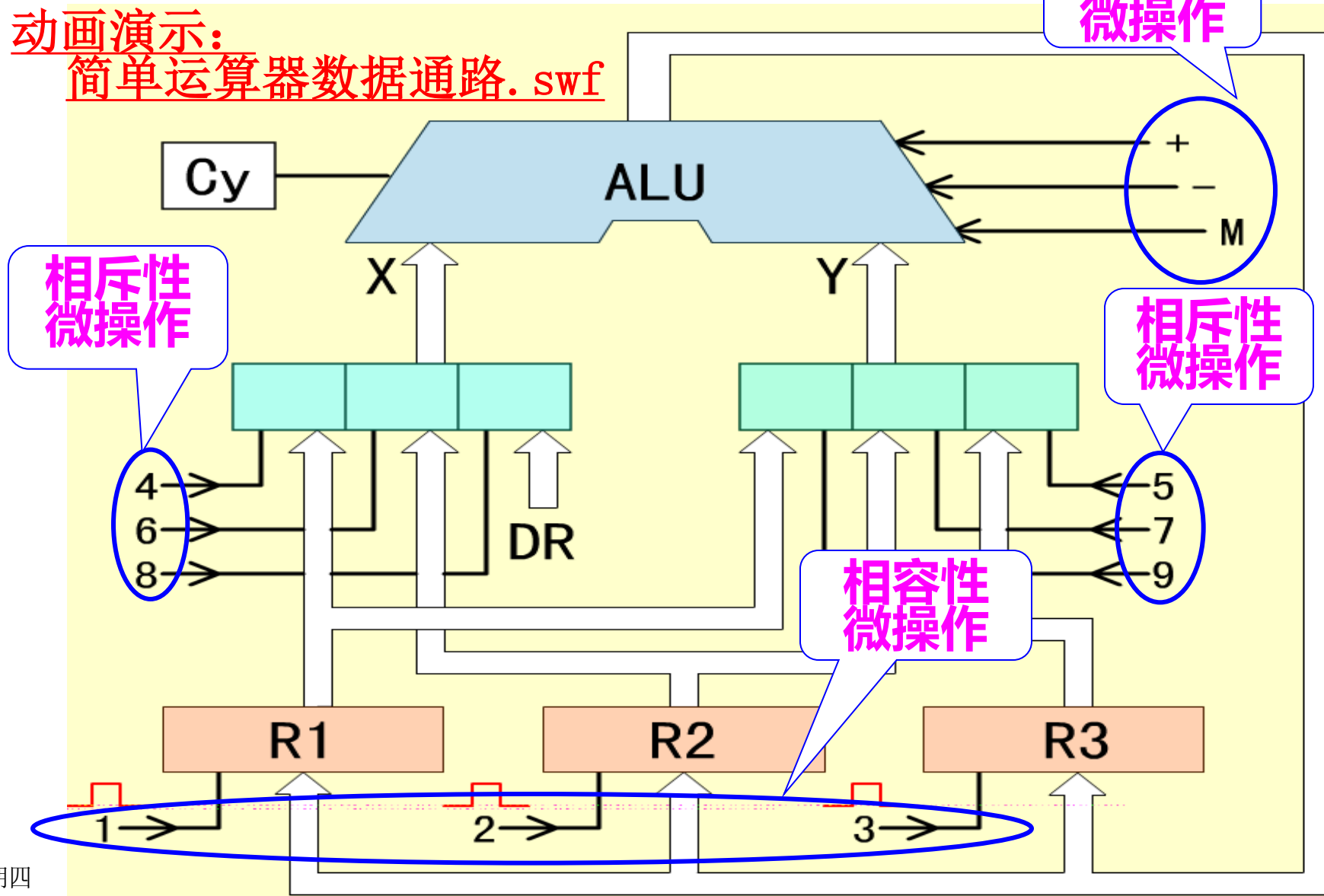
- 执行部件通过反馈线向控制部件反映当前操作状态，以使控制部件决定下一步的新微命令；

# >>> 微操作的分类

- 由于数据通路的结构关系，微操作可分为相容的和互斥的两种：
- 相斥性微操作
  - 是指不能同时或不能在同一个节拍内并行执行的微操作。  
可以编码
  - 例如取指周期中  $((PC) \rightarrow MAR; 1 \rightarrow R$  (内存读命令) )
- 相容性微操作
  - 是指能够同时或在同一个节拍内并行执行的微操作。必须各占一位



动画演示:  
简单运算器数据通路.swf





## 2、微指令和微程序

### ➤ 微指令

- 在机器的一个CPU周期内，一组实现一定操作功能的微命令的组合，构成一条微指令

- 它是微命令的组合，微指令存储在控制器中的控制存储器

### ➤ 微指令一般包含操作控制和顺序控制两大部分信息

- 操作控制字段：又称微操作码字段，用以产生某一步操作所需的各个微操作控制信号；

- ◇ 某位为1，表明发微指令
- ◇ 微指令发出的控制信号都是节拍电位信号，持续时间为一个CPU周期
- ◇ 微命令信号还要引入时间控制

- 顺序控制字段：又称微地址码字段，用于控制产生下一条微指令的地址；

- 所有的微指令都存放于控制存储器中，使用地址访问；

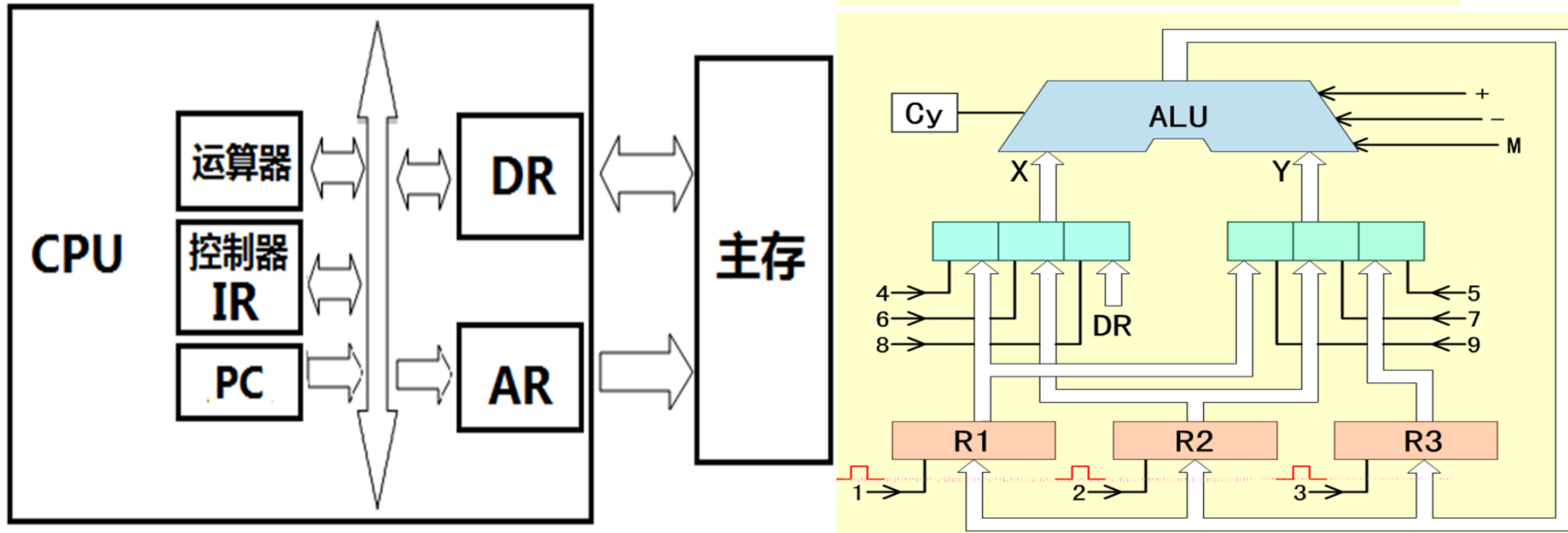
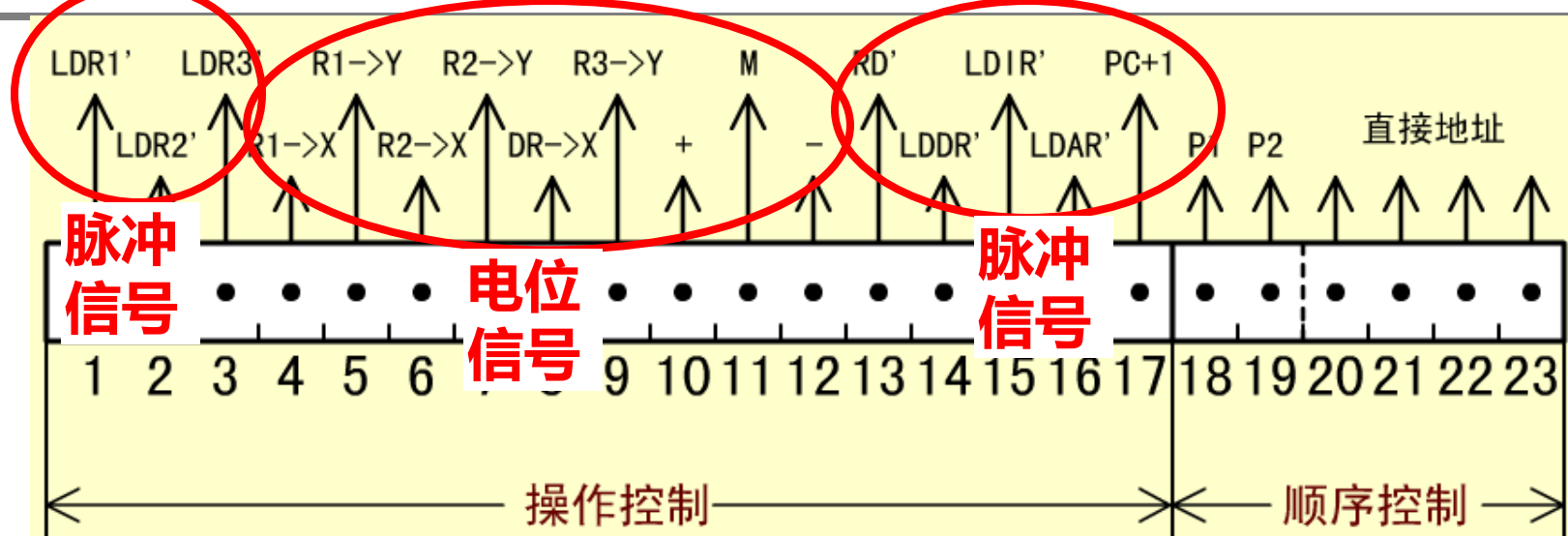
微地址

## >>> 2、微指令和微程序

### ➤ 微程序

- 一系列微指令的有序集合就是微程序，每条机器指令都对应着一段微程序；
  - ◆ 微地址：存放微指令的控制存储器的单元地址
  - ◆ 一段微程序对应一条机器指令。

# >>> 微指令基本格式



微指令格式对照的通路





# 微指令基本格式

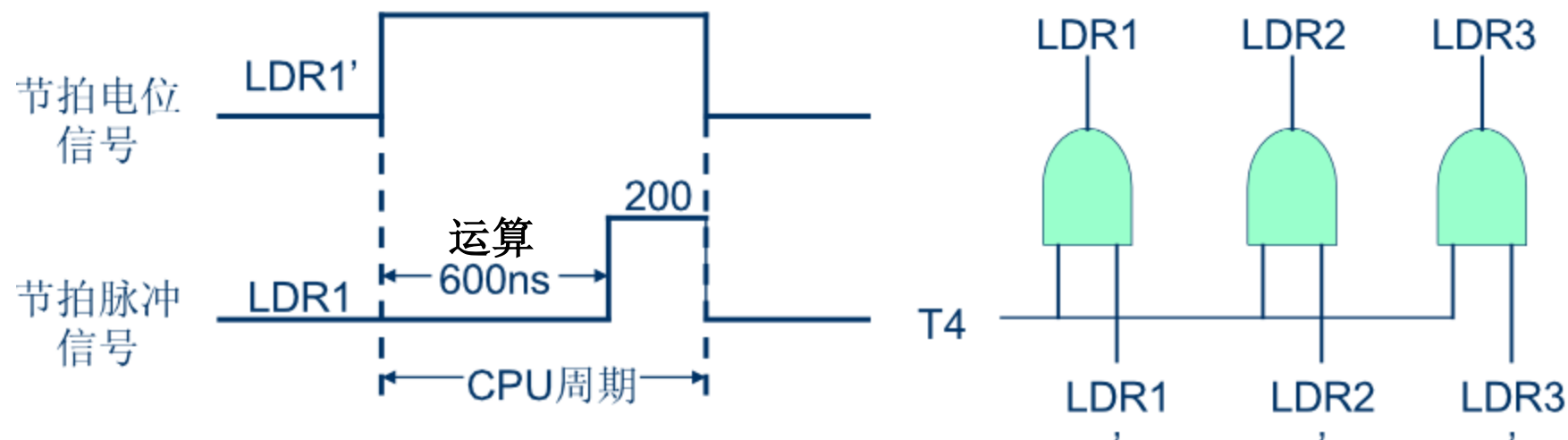
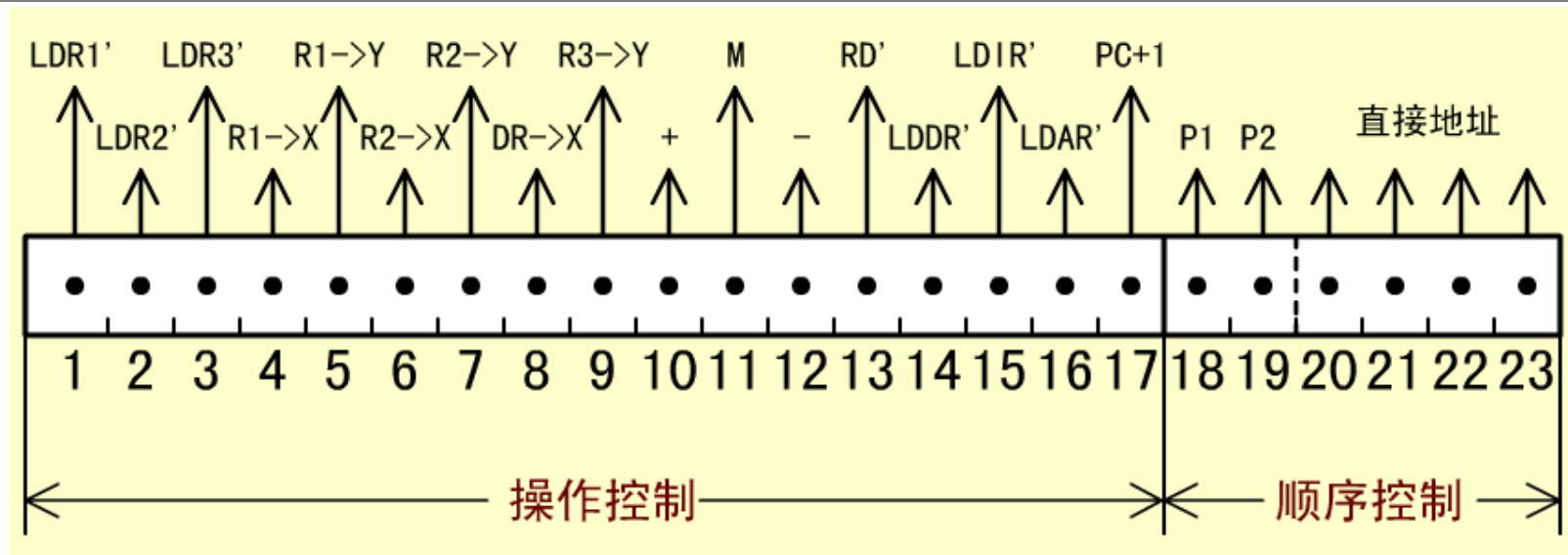


图5.23 运算器操作时序与产生逻辑

## >>> 3、微程序控制器原理框图

➤ 控制存储器（ $\mu\text{CM}$ ），微程序控制器的核心部件，用来存放微程序。

- 存放实现指令系统所需要的全部微指令；
- 由只读存储器构成，要求速度快，读出周期短。

➤ 微指令寄存器( $\mu\text{IR}$ )

- 存放当前正在执行的一条微指令；它的位数同微指令字长相等。
- 由微地址寄存器和微命令寄存器两部分构成；

◆ 微地址寄存器：用来产生初始微地址和后继微地址，以保证微指令的连续执行。

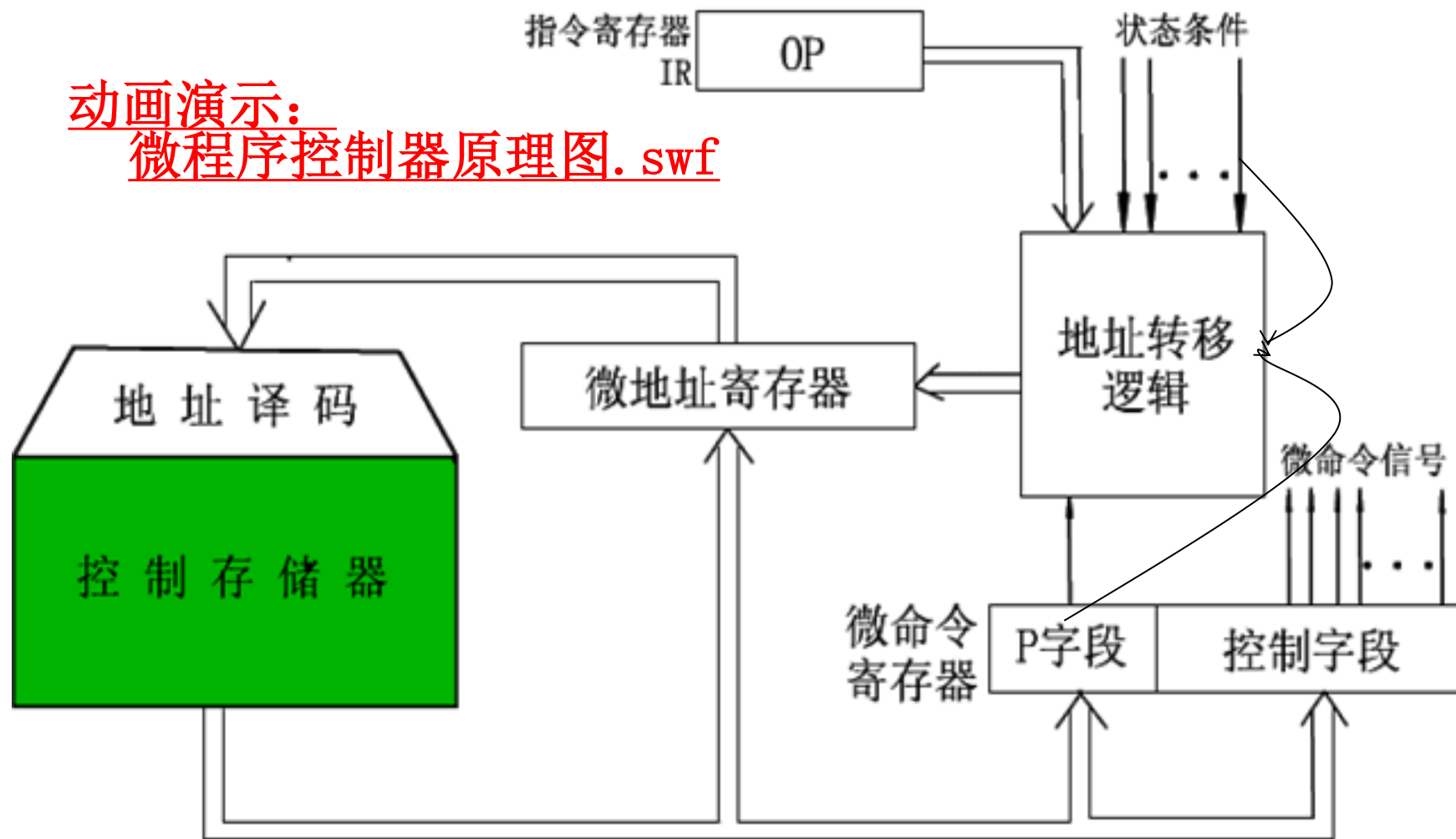
◆ 微命令寄存器( $\mu\text{MAR}$ )：保存一条微指令的操作控制字段和判别测试字段的信息，它接受微地址形成部件送来的微地址，为下一步从 $\mu\text{CM}$ 中读取微指令作准备。

➤ 地址转移逻辑

- 跳跃寻址微指令时，负责微地址的修改。比如结束当前指令的最后一条微指令，转向下一条机器指令的第一条取指微指令

# >>> P166图5.24 微程序控制器原理框图

动画演示：  
[微程序控制器原理图.swf](#)





# 微程序控制器的工作过程

- “取指微指令” 为所有指令的公用微指令；
  - 通常放在控制存储器中的 “0”地址单元；
  - 所有机器指令的**最后一条微指令的直接地址**都指向0地址单元，用以取下一条微指令；
- **1、机器指令**操作码译码，转入执行周期对应微程序（自动）；
  - 操作码译码，通过微地址形成部件产生出该机器指令所对应的微程序的入口地址，即**P1测试**，将所要执行微指令的地址送入微地址寄存器；
- **2、从μCM**逐条读取微指令并执行，每条微指令都能自动产生下一条微指令的地址。
  - 一般顺序寻址，直接使用下址字段，寻址后续微指令；
  - **需要跳跃寻址的，经P2条件测试**，修正微指令的下址。
- **3、**执行完对应于一条机器指令的微程序后，返回到取指微指令，不断重复，直至程序执行完毕。

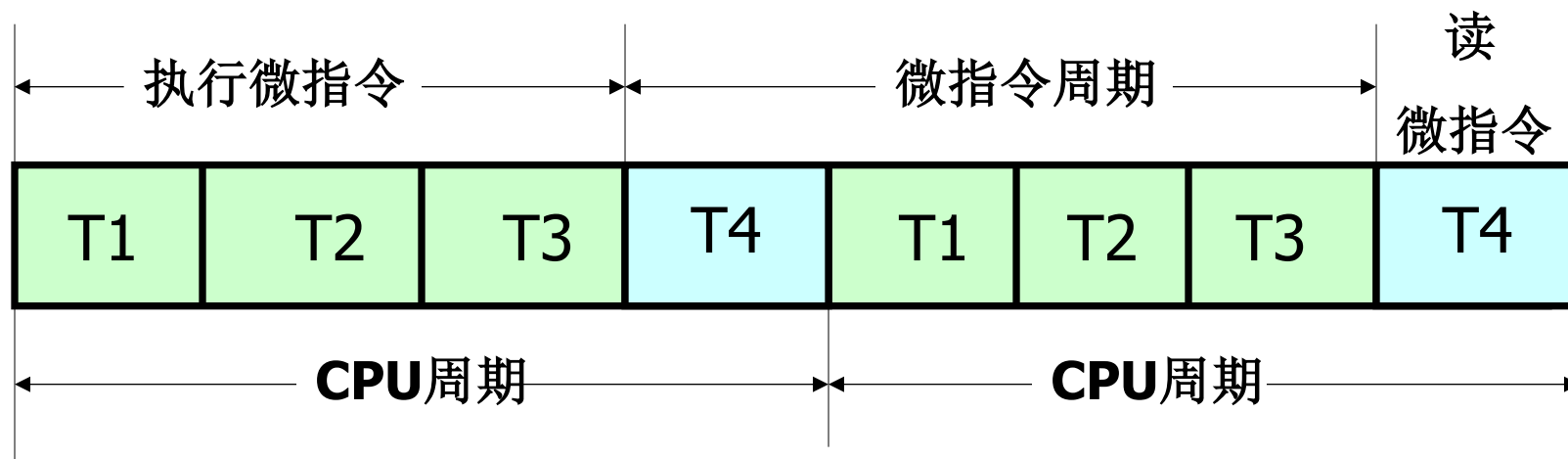
## >>> 5、CPU周期与微指令周期的关系

### ➤ 微指令周期

- 读出微指令的时间加上执行该条微指令的时间；
- 这是串行方式的微程序控制器中的概念；

➤ 为了确保机器的控制信号的同步，一个微指令周期时间设计得恰好和**CPU**周期时间相等。

## >>> CPU周期与微指令周期关系的例子



➤ **CPU周期为0.8us，包括4个节拍脉冲T1~T4，每个脉冲200ns；**

○ T1—T3的600ns为执行微指令的时间；

○ T4的200ns为取微指令的时间；

➤ **T4的上升沿保存微指令的执行结果，T4期间取微指令。**

➤ **T1的上升沿，将读出的微指令存入微指令寄存器。**

## >>> 6、机器指令与微指令的关系

➤ 一条机器指令对应一个微程序，一个微程序是由若干条微指令组成的；

○ 一条机器指令所完成的操作划分成若干条微指令来完成，由微指令进行解释和执行。

动画演示：

机器指令与微指令的关系.swf

○ 每一个CPU周期对应一条微指令。

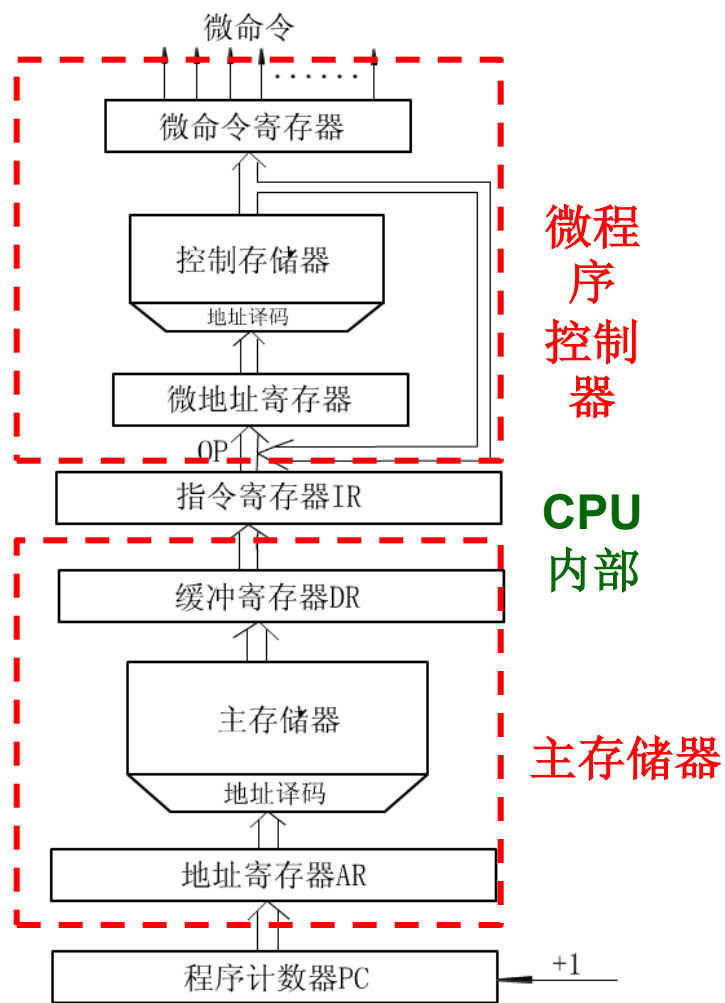
➤ 机器指令与微指令的关系

○ 使用机器指令编写的程序存放于系统的主存储器中，使用存储单元地址访问；

○ 每条机器指令对应的微程序，是由保存在控制存储器的微指令构成的，使用微地址来访问；

## >>> 6、机器指令与微指令的关系

- 一条机器指令对应一个微程序，一个微程序由若干条微指令组成的；
- 一条机器指令根据指令功能和硬件结构，划分成若干个操作，每个操作由一条微指令解释和执行。
- 每个操作对应一个CPU周期（一条微指令）。
- 机器指令与微指令的关系
- 机器指令编写的程序存放于系统主存中，使用存储单元地址访问；
- 系统的微指令存放于CPU内部的控制存储器中，机器指令执行时解释为微程序，利用微地址访问控存逐条获取；





## >>> 5.4.2 微程序设计技术

---

微程序设计的关键是如何确定微指令的结构。设计微指令应当追求的目标

- 有利于缩短微指令的长度
- 有利于缩小CM的容量
- 有利于提高微程序的执行速度
- 有利于对微指令的修改
- 有利于提高微程序设计的灵活性

## >>> 5.4.2 微程序设计技术

---

1. 微命令编码
2. 微地址的形成方法
3. 微指令格式
4. 动态微程序设计

# >>> 1、微命令编码

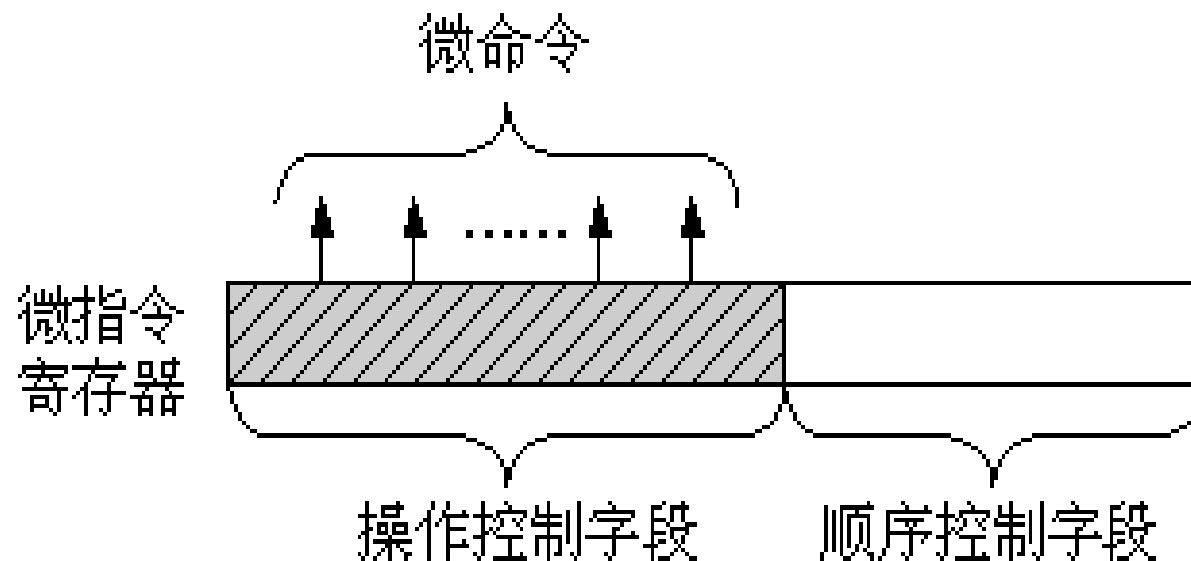
- 指微指令中**操作控制字段**的编码表示方法，以及如何把编码翻译成相应的微指令。
- 微命令编码主要考虑的问题：
  - 如何有效缩短微指令字长；
  - 如何有利于缩短微程序，减少所需的控存空间；
  - 如何有利于提高微程序执行速度；
- 微命令编码表示方法
  - 直接表示法**、**编码表示法**、混合表示法（自学）

# >>> 1、微命令编码

## ➤ 微命令编码表示方法

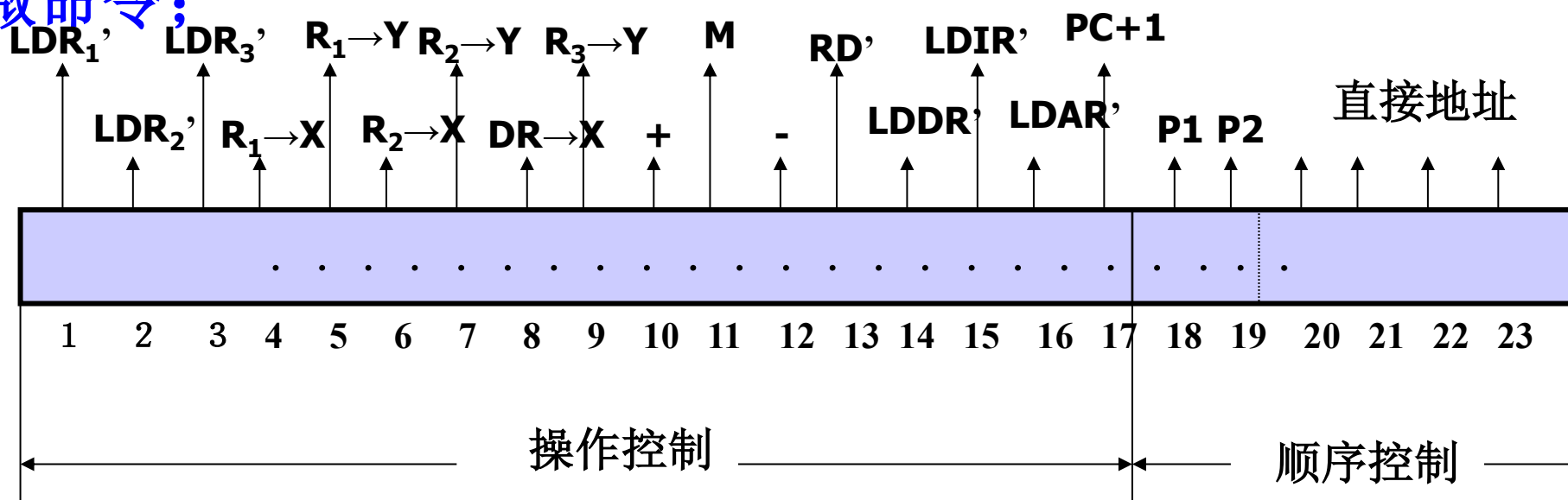
○ 直接表示法、编码表示法、混合表示法

➤ 直接表示法：操作控制字段中的各位分别可以直接控制计算机，不需要进行译码。



# >>> 直接表示法

- 操作控制字段中的每一位代表一个微命令，一条微指令能定义多个可并行的微命令；



- **原则**：相容性微命令，可以并行完成的微命令

- **优点**

- 简单直观，其输出可直接用于控制，执行速度快（同时可进行多个微操作）；

- **缺点**

# >>> 编码表示法

## ➤ 编码方法

- 将微指令操作控制字段划分为若干个**子字段**；每个字段是一组**相斥微命令**。
- 每个子字段的所有微命令进行统一编码，每段内采用最短编码法；
- 每个子字段的**不同编码表示不同的微命令**；
- 段与段之间采用直接控制法

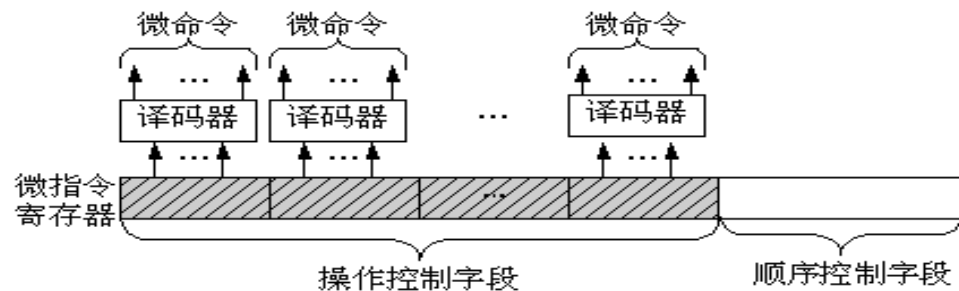
## ➤ 例如，某机器指令系统总共需要**256个微命令**

### ○ 采用直接编码法

- ◆ 微指令的操作控制字段需**256位**；

### ○ 采用编码表示法

- ◆ 若采用4位一个子字段，则每个子字段可编码**16个微命令**；
- ◆ 256个微命令需**16个子字段**，即微指令的操作控制字段仅**64位**。



# >>> 编码表示法

## ➤ 遵循原则：

- 把**相斥的微命令**划分在同一个字段中，相容的微命令划分在不同字段；
- 字段的划分应与数据通路结构相适应；
- 每个子字段应留出一个**空操作状态**，即可选择不发出该子字段对应的所有微命令；  
例如3位二进制位译码后可表示7个微命令。
- 每个子字段所定义微命令不宜太多；

## ➤ 优点

- 可大大缩短微指令字长；

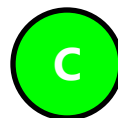
## ➤ 缺点

- **需要微命令译码**，增加译码电路。故微程序的执行速度稍稍减慢。

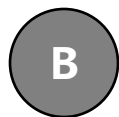
某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，共有33个微命令，构成5个互斥类，分别包含7、3、12、5和6个微命令，则操作控制字段至少有（ ）



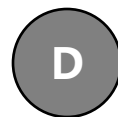
A 5位



C 15位



B 6位



D 33位

7---3, 3---2, 12---4, 5---3, 6---3

至少:  $3+2+4+3+3=15$

提交



## >>> 2、微地址的形成方法

- 微指令执行的顺序控制问题，实际上就是如何确定下一条微指令的地址问题。
- 微程序的入口地址
  - 微程序的第一条微指令所在控存单元的地址；
    - ◆ 确切的说，应该是**机器指令执行周期**的第一条微指令的地址；
    - ◆ 当公用的取指微程序从主存中取出机器指令之后，由机器指令的操作码字段指出各段微程序的入口地址，这是一种多分支(或多路转移)的情况。
- 产生后继地址的两种方法
  - 计数器的方式
  - 多路转移的方式

# >>> 计数器方式

➤同CPU中程序计数器产生机器指令地址的方法相类似。

➤方法：

- 微程序顺序执行时，其后继微地址就是现行微地址加上一个增量(通常为1)；
- 当微程序遇到转移或转子程序时，由微指令的转移地址段来形成转移微地址。
- 在微程序控制器中也有一个微程序计数器 $\mu PC$ ，一般情况下都是将微地址寄存器 $\mu MAR$ 作为 $\mu PC$

➤优点

- 微指令的顺序控制字段较短，微地址产生机构简单。

➤缺点

- 多路并行转移功能较弱，速度较慢，灵活性较差。

# >>> 多路转移方式

- 一条微指令具有多个转移分支的能力成为多路转移。
- 可根据条件信息选定某一个候选微地址的方法。
  - 当微程序不产生分支时，后继地址就是顺序控制字段的地址
  - 当微程序产生分支，按照顺序控制字段的“判别标志”和“状态条件”信息来选择其中一个微地址
    - ◆ 状态条件有 $n$ 位标志位，则有路 $2^n$ 转移
- 特点
  - 能以较短的顺序控制字段配合，实现多路并行转移，灵活性好，速度较快；
  - 但转移地址逻辑需要用组合逻辑方法设计。

➤➤➤ 【例】已知某计算机采用微程序控制方式，其控存容量为 $512 \times 32$ 位，微程序可以在控存中实现转移，可控制微程序转移的条件有6个，采用直接编码方式，后继微指令地址采用多路转移方式。

- 微指令字长32位，格式如下，请说明微指令中3个字段分别应为多少位。

|       |      |    |
|-------|------|----|
| 微操作编码 | 测试指令 | 下址 |
|-------|------|----|

- ① 由控存单元数可知共512个，则每个控存单元地址位为9位，则微指令中的后继微指令地址位数为9位；
- ② 可控制微程序转移的条件为6个，且按照直接控制编码，则测试条件位数为6位；
- ③ 剩下的为操作控制字段可用位数 $32 - 9 - 6 = 17$ 位；

## >>> 3、微指令格式

➤影响微指令格式的主要因素是微指令的编译方法，通常考虑速度、成本等原因，设计时采用不同的编译方法，因此编译方式也不同

|      |            |       |
|------|------------|-------|
| 控制字段 | 判别<br>测试字段 | 下地址字段 |
|------|------------|-------|

➤两类：水平型微指令和垂直型微指令

➤水平型微指令(例如本书的例子)

○一次能定义并执行多个并行操作微命令的微指令；

◆一般由操作控制字段、判别测试字段、下地址字段三部分构成；

○根据控制字段编码方式的不同，可分为全水平型、字段译码法水平型、直接和译码相混合的水平型三种微指令。

## >>> 3、微指令格式

- 垂直型微指令（类似机器指令结构），采用编码方式。
  - 微指令中设置微操作码字段，采用微操作码编译法，由微操作码规定微指令的功能；
  - 设置微操作控制字段时，一次只能执行一到二个微命令的微指令称为垂直型微指令。



## >>> 水平型微指令与垂直型微指令比较

1. 水平型微指令**并行操作能力强**，效率高，灵活性强。垂直型微指令则较差。
2. 水平型微指令**执行一条指令的时间短**，垂直型微指令执行时间长。
3. 由水平型微指令解释指令的微程序，具有**微指令字比较长，但微程序短**的特点。  
垂直型微指令则相反，微指令字比较短而微程序长。
4. 水平型微指令用户**难以掌握**，而垂直型微指令与指令比较相似，相对来说，比较容易掌握（因为可编程）。

下列关于主存储器(MM)和控制存储器(CS)的叙述中，错误的是

---

- ☐ A MM在CPU外，CS在CPU内
- ☒ B MM按地址访问，CS按内容访问
- ☐ C MM存储指令和数据，CS存储微指令
- ☐ D MM用RAM和ROM实现，CS用ROM实现



## 5.6 流水CPU

---

**5.6.1 并行处理技术**

**5.6.2 流水CPU的结构**

**5.6.3 流水线中的主要问题**

## >>> 5.6.1 并行处理技术

### ➤ 并行性的两种含义：

① **同时性**：两个以上事件在同一时刻发生；

◆ 如多机系统中，同一时刻多个进程在运行。

② **并发性**：两个以上事件在同一间隔内发生。

◆ 如并发程序，某一时刻CPU中只有一个进程在运行，而在一个时间段内，多个进程同时运行。

### ➤ 并行性的三种形式：

○ **时间并行（重叠）**：让多个处理过程在时间上相互错开，轮流使用同一套硬件设备的各个部件，以加快硬件周转而赢得速度，实现方式就是采用流水处理部件。

○ **空间并行（资源重复）**：以数量取胜，设置重复资源，同时工作。

◆ 它能真正的体现同时性；LSI和VLSI为其提供了技术保证

○ **时间并行 + 空间并行**：时间重叠和资源重复的综合应用。

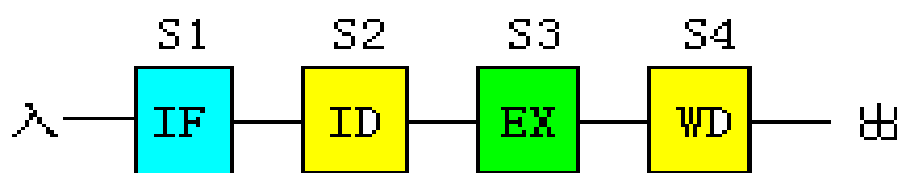
## >>> 2、流水CPU的时空图

➤实现流水：任务分割成一系列子任务，子任务在流水线的各个阶段并发地执行。

➤流水线CPU时空图

- IF (Instruction Fetch取指)
- ID (Instruction Decode指令译码)
- EX (Execution执行或计算有效地址)
- WB (Write Back写回)

➤指令执行过程划分为不同行。



多条指令同时进行

(a) 一个指令流水线过程段

## >>> 2、流水CPU的时空图

### ➤ 流水线基本原理

#### ○ 顺序方式的串行指令执行过程：

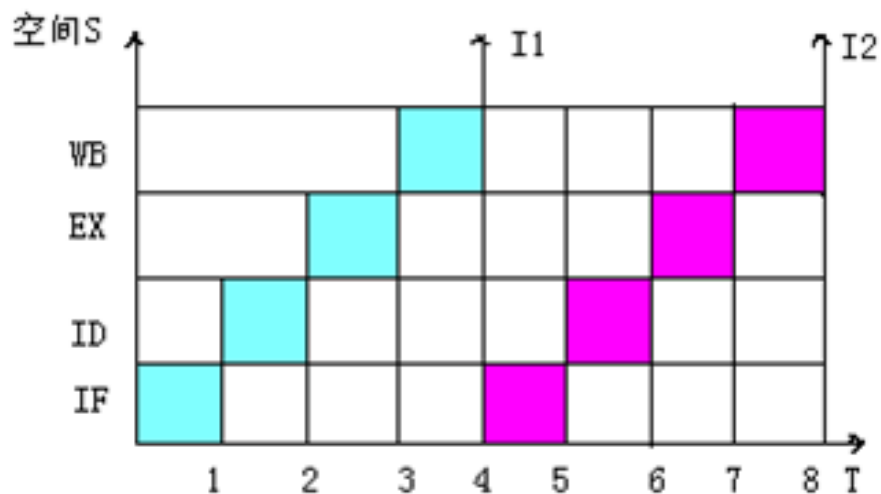
|     |     |       |          |     |     |       |     |
|-----|-----|-------|----------|-----|-----|-------|-----|
| 取指1 | 译码1 | 取操作数1 | 运算并保存结果1 | 取指2 | 译码2 | 取操作数2 | ... |
|-----|-----|-------|----------|-----|-----|-------|-----|

#### ○ 将一条指令的执行过程分成4段，并行执行过程：

|    |    |      |         |         |         |         |
|----|----|------|---------|---------|---------|---------|
| 取指 | 译码 | 取操作数 | 运算并保存结果 |         |         |         |
|    | 取指 | 译码   | 取操作数    | 运算并保存结果 |         |         |
|    |    | 取指   | 译码      | 取操作数    | 运算并保存结果 |         |
|    |    |      | 取指      | 译码      | 取操作数    | 运算并保存结果 |

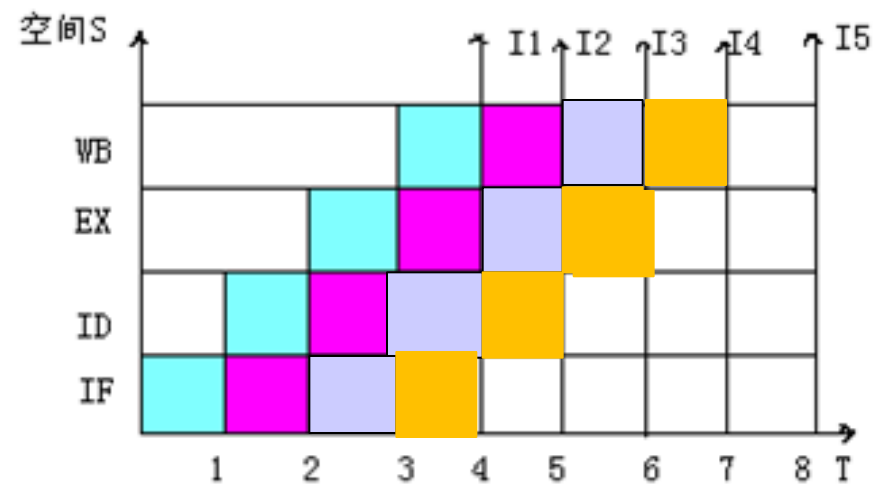
一般流水线时钟周期应选取4段中最长的时间

## >>> 5.6.2 流水CPU的结构



(b) 非流水线时空图

非流水CPU

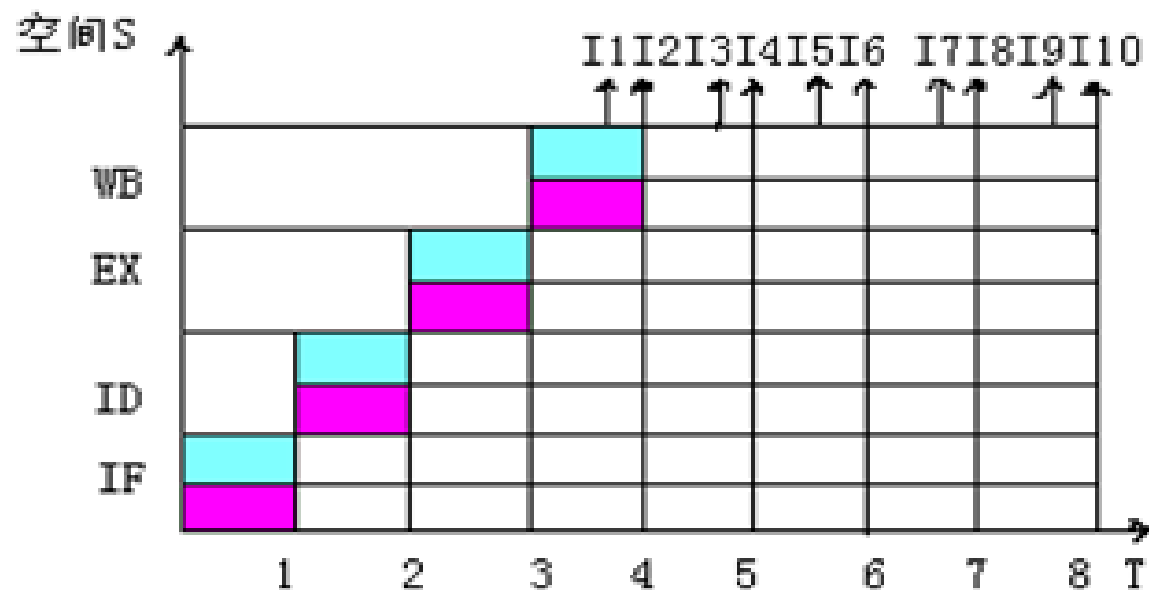


(c) 标量流水线时空图

流水CPU

流水时空图主要分析流水线的性能

## >>> 5.6.2 流水CPU的结构



(d) 超标量流水线时空图

- 具有两条以上的指令流水线
- 上图中流水线满载时，每一个时钟周期可以执行2条指令
- 采用时间和空间并行技术
- 需要配置多个功能部件，比如多个运算器（超标量）
- 实现并行的方法可依靠编译优化技术，把可并行执行的指令搭配并行执行（超标量）

# >>> 流水CPU的加速比

➤ 若某**CPU**指令执行，可划分为**K**个阶段，每个阶段**1**个周期；

➤ 非流水线**CPU**处理**n**个任务时，所需时钟周期数

$$T_1 = n \times k$$

➤ 一个具有**k**级过程段的流水**CPU**处理这**n**个任务时，需要的时钟周期数

$$T_k = k + (n - 1)$$

○ **k**个时钟周期用于处理第一个任务；

○ **k**个周期（即第一个任务）后，流水线被装满，剩余的**n - 1**个任务只需**n - 1**个周期即可完成（即每个任务只需要一个周期，剩余共**n-1**个任务）。

➤ 将**T<sub>1</sub>**和**T<sub>k</sub>**的比率定义为**k**级线性流水处理器的加速比：

$$C_k = T_1 / T_k$$

# >>> 指令流水线的性能指标

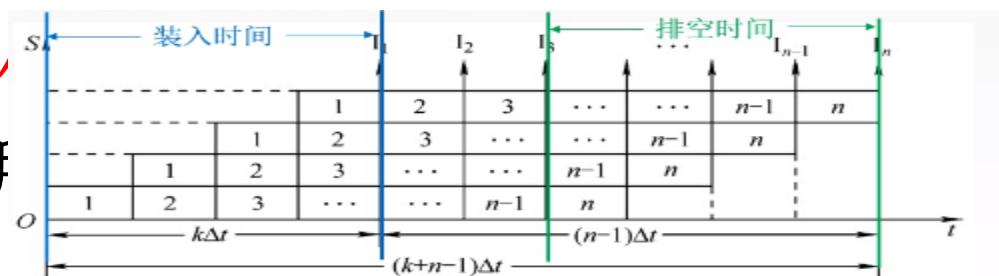
- 一条**k**段线性流水线完成**n**个任务，假设每一段完成时间均为 $\Delta t$ 。一般取 $\Delta t$ =一个时钟周期

- 吞吐率**TP**：单位时间内流水线完成的<sup>n</sup>任务数量，或是输出的结果数量。

$$TP = \frac{n}{(k + n - 1)\Delta t}$$

- 加速比**S**：完成同一批任务， $S = \frac{\text{不使用流水线所用时间}}{\text{使用流水线所用时间}}$ 之比。

- 效率**E**：在时空图上，完成**n**个<sup>n</sup>任务所用时间、**k**个流水段理想情况下其极限值 $E_{\max}=1$ 。



一条指令的执行分为**k**个阶段，每个阶段耗时 $\Delta t$ ，一般取 $\Delta t$ =一个时钟周期



# >>> 3、流水线分类

## ➤ 指令流水线

- 指令执行的并行处理；
- 指令流划分为取指、译码、取操作数、执行、写回等过程；

## ➤ 算术流水线

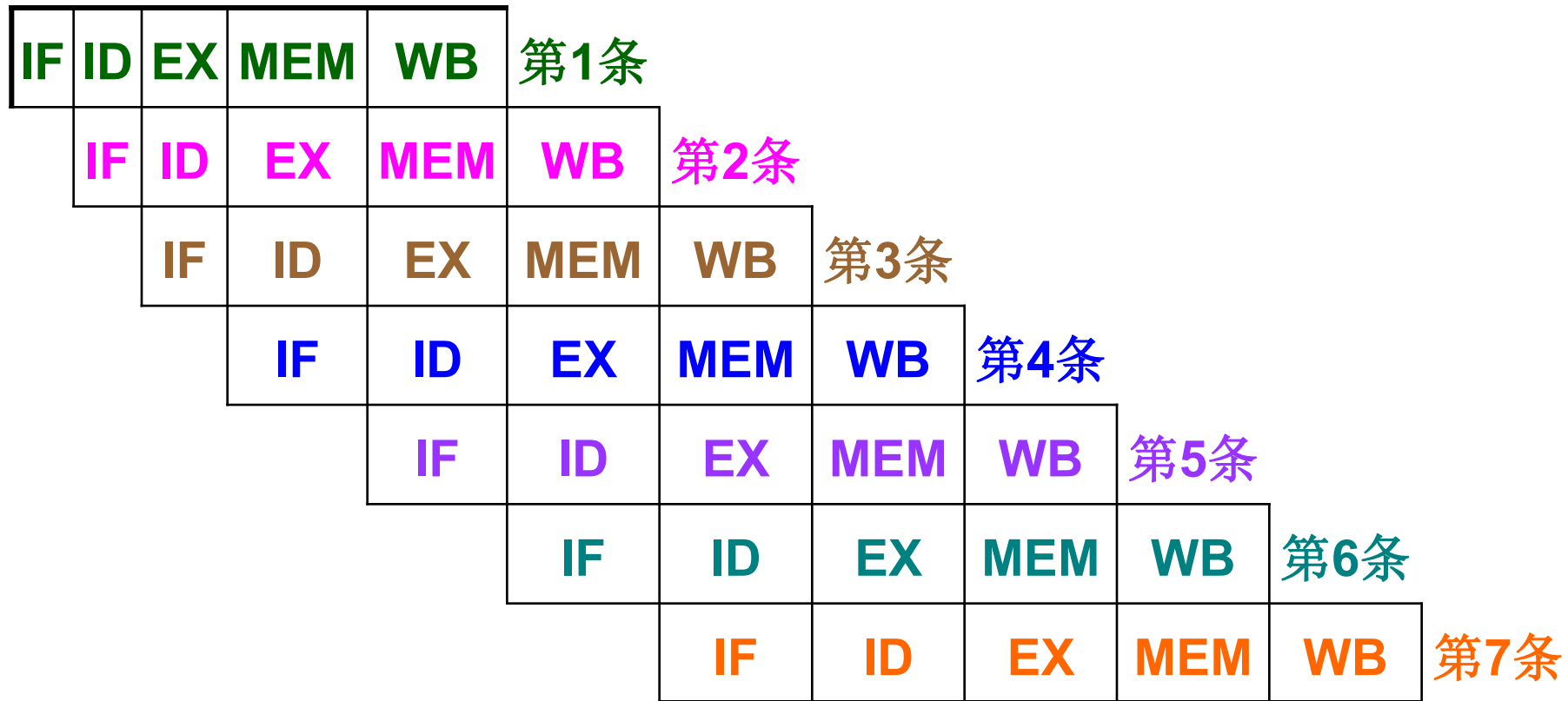
- 运算操作步骤的并行处理；（比如浮点数的加法操作分成对阶、尾数相加、结果规格化等过程）
- 现代微机中大多采用流水的算术运算器；

## ➤ 处理机流水线

- 程序步骤的并行处理；
- 将每一阶段的处理分散在不同的机器上，应用于多机系统中；

➤➤➤ 【例1】指令流水线有取指(IF)、译码(ID)、执行(EX)、访存(MEM)、写回寄存器堆(WB)五个过程段，共有7条指令连续输入此流水线，时钟周期为100ns。

➤请画出流水处理的时空图；



➤➤➤【例1】指令流水线有取指(IF)、译码(ID)、执行(EX)、访存(MEM)、写回寄存器堆(WB)五个过程段，共有7条指令连续输入此流水线，时钟周期为100ns。

➤求流水线的实际吞吐率（单位时间里执行完毕的指令数）；

○流水线执行完7条指令的时间是

$$5 + (7 - 1) = 11 \text{ 个时钟周期}$$

○故实际吞吐率为

$$7 / (11 \times 100\text{ns}) \approx 64 \times 10^5 \text{ 条指令/s}$$

➤求流水处理器的加速比。

○K级流水线处理器的加速比为

$$C_k = \text{非流水时间} T_1 / \text{K级流水时间} T_k = n \times k / (k + (n - 1))$$

○本题中，已知数据  $n = 7$ ， $k = 5$ ，则

$$C_k = 7 \times 5 / (5 + 6) = 35 / 11 \approx 3.18$$

某CPU主频为1.03GHz，采用4级指令流水线，每个流水段的执行需要1个时钟周期。假定CPU执行了100条指令，在其执行过程中，没有发生任何流水线阻塞，此时流水线的吞吐率为（ ）

- ☐ A  $0.25 \times 10^9$ 条指令/秒
- ☒ C  $1.0 \times 10^9$ 条指令/秒
- ☐ B  $0.97 \times 10^9$ 条指令/秒
- ☐ D  $1.03 \times 10^9$ 条指令/秒

提交



某CPU主频为1.03GHz，采用4级指令流水线，每个流水段的执行需要1个时钟周期。假定CPU执行了100条指令，在其执行过程中，没有发生任何流水线阻塞，此时流水线的吞吐率为（ ）

**解答：**采用4级流水执行100条指令，在执行过程中共用 $4 + (100 - 1) = 103$ 个时钟周期，CPU的主频1.03GHz，也就是每秒钟有1.03G个时钟周期。流水线的吞吐率 $= 1.03G \times 100 / 103 = 1.0 \times 10^9$ 条指令/秒

## >>> 5.6.3 流水线中的主要问题

### ➤ 资源相关

- 多条指令进入流水线后，在同一机器周期内争用同一个功能部件所发生的冲突；
- **解决方法**：指令推迟执行，或是设置重复资源；

### ➤ 数据相关

- 在一个程序中，如果必须等前一条指令执行完毕后，才能执行后一条指令，这两条指令就是数据相关；
- **解决方法**：指令推迟执行，或是定向传送技术。

### ➤ 控制相关

- 当执行转移指令时，根据转移条件是否发生来控制指令的执行顺序；
- **解决方法**：延迟转移法、转移预测法。

## >>> 5.6.3 流水线中的主要问题

### ➤ 资源相关

- 多条指令进入流水线后，在同一机器周期内争用同一个功能部件所发生的冲突；
- **解决方法：**指令推迟执行，或是设置重复资源；

| 指令 \ 时钟   | 1  | 2  | 3  | 4   | 5   | 6   | 7   | 8   |
|-----------|----|----|----|-----|-----|-----|-----|-----|
| I1 (Load) | IF | ID | EX | MEM | WB  |     |     |     |
| I2        |    | IF | ID | EX  | MEM | WB  |     |     |
| I3        |    |    | IF | ID  | EX  | MEM | WB  |     |
| I4        |    |    |    | IF  | ID  | EX  | MEM | WB  |
| I5        |    |    |    |     | IF  | ID  | EX  | MEM |



## 5.6.3 流水线中的主要问题

### ➤ 数据相关

- 在一个程序中，如果必须等前一条指令执行完毕后，才能执行后一条指令，这两条指令就是数据相关；
- RAW(Read After Write)
  - ◆ 后面指令用到前面指令所写的数据
- WAW(Write After Write)
  - ◆ 两条指令写同一个单元
  - ◆ 在简单流水线中没有此类相关，因为不会乱序执行
- WAR(Write After Read)
  - ◆ 后面指令覆盖前面指令所读的单元
  - ◆ 在简单流水线中没有此类相关
- **解决方法：**指令推迟执行，设置相关的直接通路(forwarding)，或是定向传送技术。





## 5.6.3 流水线中的主要问题

### ➤ 控制相关

- 引起原因：当执行转移指令时，根据转移条件是否发生来控制指令的执行顺序；
- 根据当前的状态条件，或顺序取下一条指令执行，或要转移到新的目标地址去取指令执行。

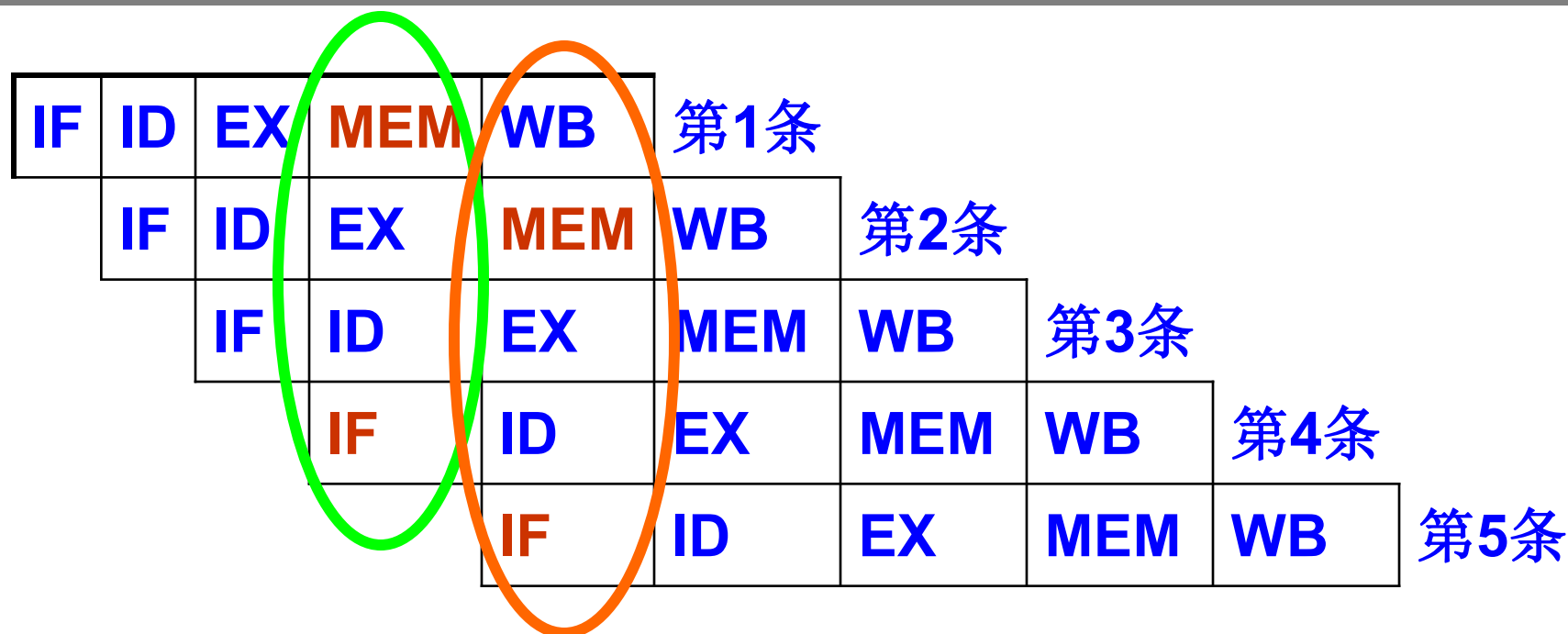
### ➤ 例如：当执行指令A的时候，紧跟A之后的几条指令已经被放到了指令流水线中且进行了预处理

- 当执行完转移指令，应跳转到新的目标地址去取指令执行，只能先把已经进入到流水线的指令清空，然后才能把转移跳转后所需的指令取过来放在流水线上，但此时流水线就会出现断流

### ➤ 解决方法：

- 延迟转移法（将已经到达流水线的指令先执行再转移）
- 转移预测法（硬件实现）

# >>> 资源相关



- 第1、2条指令的MEM与第4、5条指令的IF，都需要访存；
- 若数据与指令在同一存储器中，且单端口访问，则争用资源；
- 解决方法
  - ① 第4条指令推迟一个时钟周期；但会引起流水线的断流
  - ② 将数据与指令存储器分离；或采用双端口存储器；

# >>> 数据相关

➤ 设某程序段如右

```
ADD  R1, R2, R3  ; (R2) + (R3) → R1
SUB  R4, R1, R5   ; (R1) - (R5) → R4
AND  R6, R1, R7   ; (R1) ∧ (R7) → R6
```

➤ 以上3条指令的流水执行如下

|    |    |    |     |     |     |     |     |
|----|----|----|-----|-----|-----|-----|-----|
| IF | ID | EX | MEM | WB  | ADD |     |     |
|    | IF | ID | EX  | MEM | WB  | SUB |     |
|    |    | IF | ID  | EX  | MEM | WB  | AND |

- 在第5个时钟周期ADD指令才会更新R1内容;
- 但第4、5个时钟周期SUB、AND指令就需要读取R1的内容;

➤ 解决方法

- ① 数据相关指令推迟若干时钟周期执行;
- ② 数据定向 (旁路) 技术, 直接采用运算器的结果;
- ③ 改变程序的执行顺序, 将无关指令提前执行;

不一定总有效

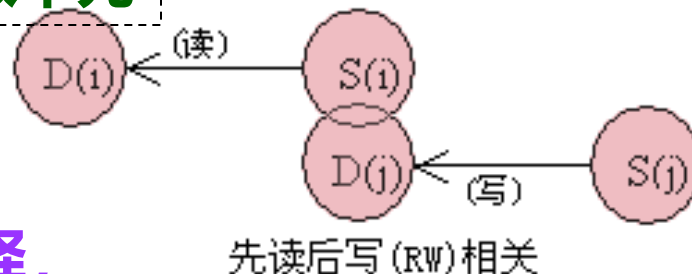


# 数据相关的类型

设指令i、j依次流入流水线

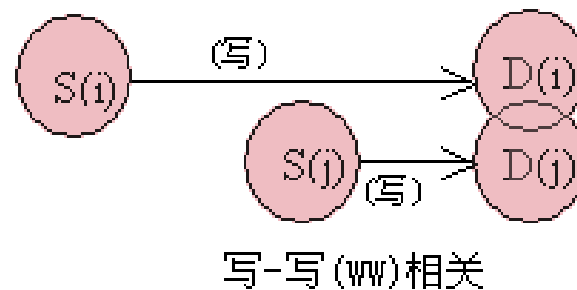
## ➤ “先读后写” WAR相关 i是读单元是j写数单元

- 正常顺序是：i先读，j再写；
- 由于异步流动下，j可能先于指令i被解释，从而i读到j写后的数据，不正确。

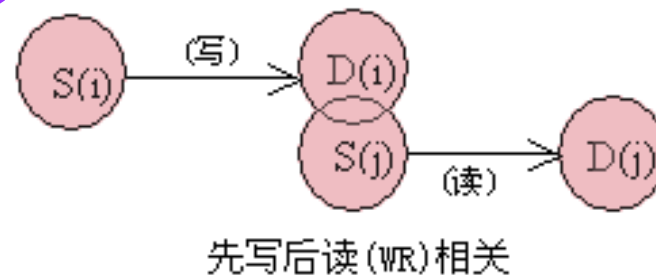


## ➤ “写-写” 相关 i、j写同一单元

- 正常情况下，最后单元中保存j写的数据；
- 由于异步流动，j可能先于i写数据，使该单元最后结果不正确。



## ➤ “先写后读” RAW相关类似





## 课本P179 【例5.4】

流水线中有三类数据相关冲突：**写后读(RAW)相关**；**读后写(WAR)相关**；**写后写(WAW)相关**。判断以下三组指令各存在哪种类型的数据相关。

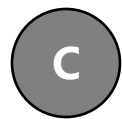
- ① I1    **ADD R1,R2,R3 ; (R2+R3) → R1**    **写后读(RAW)相关：**  
I2    **SUB R4,R1,R5 ; (R1-R5) → R4**  
I1指令应先写入R1,由于I2指令进入流水线,变成I2指令在I1指令写入前就读出R1内容
- ② I3    **STA M(x), R3; (R3) → M(x),M(x)** 是存储单元  
I4    **ADD R3,R4,R5; (R4+R5) → R3**  
**读后写(WAR)相关**
- ③ I5    **MUL R3,R1,R2; (R1) × (R2) → R3**    **写后写(WAW)相关**  
I6    **ADD R3,R4,R5; (R4+R5) → R3**

某计算机的指令流水线由四个功能段组成，指令流经各功能段的时间（忽略各功能段之间的缓存时间）分别是**90ns**、**80ns**、**70ns**和**60ns**，则该计算机的**CPU**时钟周期至少是



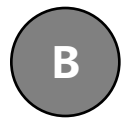
A

**90ns**



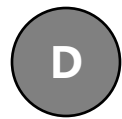
C

**70ns**



B

**80ns**



D

**60ns**

提交

(2017年) 下列关于超标量流水线特性的叙述中, 正确的是\_\_\_\_\_

- I. 能缩短流水线功能段的处理时间
- II. 能在一个时钟周期内同时发射多条指令
- III. 能结合动态调度技术提高指令执行并行性

- |                                 |                                             |
|---------------------------------|---------------------------------------------|
| <input type="radio"/> A 仅 II    | <input checked="" type="radio"/> C 仅 II、III |
| <input type="radio"/> B 仅 I、III | <input type="radio"/> D I、II和III            |

提交

下列关于指令流水线数据通路的叙述中，错误的是

---

- ☒ A 包含生成控制信号的控制部件
- ☐ B 包含算术逻辑运算部件(ALU)
- ☐ C 包含通用寄存器组和取指部件
- ☐ D 由组合逻辑电路和时序逻辑电路组合而成



(2017) 在无转发机制的五段基本流水线(取指、译码/读寄存器、运算、访存、写回寄存器)中, 下列指令序列存在数据冒险的指令对是\_\_\_\_\_。

I1:add R1, R2, R3 ;(R2)+(R3) → R1

I2:add R5, R2, R4 ;(R2)+(R4) → R5

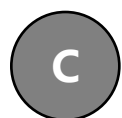
I3:add R4, R5, R3 ;(R5)+(R3) → R4

I4:add R5, R2, R6 ;(R2)+(R6) → R5



A

I1和I2



C

I2和I4



B

I2和I3



D

I3和I4